

# Decomposition pipeline for large-scale portfolio optimization with applications to near-term quantum computing

Atithi Acharya,<sup>1</sup> Romina Yalovetzky<sup>1</sup>,<sup>1</sup> Pierre Minssen,<sup>1</sup> Shouvanik Chakrabarti,<sup>1</sup> Ruslan Shaydulin,<sup>1</sup> Rudy Raymond,<sup>1</sup> Yue Sun<sup>1</sup>,<sup>1</sup> Dylan Herman,<sup>1</sup> Ruben S. Andrist,<sup>2</sup> Grant Salton<sup>1</sup>,<sup>2,3,4</sup> Martin J. A. Schuetz<sup>1</sup>,<sup>2,3</sup> Helmut G. Katzgraber<sup>1</sup>,<sup>2</sup> and Marco Pistoia<sup>1</sup>

<sup>1</sup>Global Technology Applied Research, *JPMorganChase*, New York, New York 10017, USA

<sup>2</sup>Amazon Quantum Solutions Lab, Seattle, Washington 98170, USA

<sup>3</sup>AWS Center for Quantum Computing, Pasadena, California 91125, USA

<sup>4</sup>California Institute of Technology, Pasadena, California 91125, USA



(Received 2 October 2024; accepted 4 April 2025; published 13 May 2025)

Industrially relevant constrained optimization problems, such as portfolio optimization and portfolio rebalancing, are often intractable or difficult to solve exactly. In this work, we propose and benchmark a decomposition pipeline targeting portfolio optimization and rebalancing problems with constraints. The pipeline decomposes the optimization problem into constrained subproblems, which are then solved separately and aggregated to give a final result. Our pipeline includes three main components: preprocessing of correlation matrices based on random matrix theory, modified spectral clustering based on Newman's algorithm, and risk rebalancing. Our empirical results show that our pipeline consistently decomposes real-world portfolio optimization problems into subproblems with a size reduction of approximately 80%. Since subproblems are then solved independently, our pipeline drastically reduces the total computation time for state-of-the-art solvers. Moreover, by decomposing large problems into several smaller subproblems, the pipeline enables the use of near-term quantum devices as solvers, providing a path toward practical utility of quantum computers in portfolio optimization.

DOI: [10.1103/PhysRevResearch.7.023142](https://doi.org/10.1103/PhysRevResearch.7.023142)

## I. INTRODUCTION

Portfolio optimization (PO) plays a vital role in managing vast amounts of global financial assets, requiring rapid and robust algorithms to effectively make decisions about investing capital, while managing risks. Often, PO problems include integer decision variables, which arise when the underlying assets are traded in discrete quantities. Moreover, such optimization problems impose constraints on the decision variables, such as having a fixed minimum and maximum number of selected assets, or a maximum total risk exposure of the portfolio.

Typically, PO problems can be formulated as mixed-integer programming (MIP) problems, for which a wide range of methods have been proposed. In particular, various local search methods have been proposed and successfully applied [1–3]. However, these methods often have several drawbacks when applied to MIP problems, such as scalability, the risk of getting trapped in local optima, and difficulty handling complex constraints. On the other hand, exact methods, such as those based on the *branch-and-bound* [4,5] search method, often perform better in these aspects, and are, therefore, among the most widely used tools for many hard MIP problems.

Notably, commercially available branch-and-bound based solvers offered by CPLEX [6] and GUROBI [7] have been widely adopted in the industry for optimization problems, such as traveling salesman, graph partitioning, and quadratic assignment problems [8]. These solvers employ the *branch-and-cut* procedure, which effectively combines branch-and-bound with cutting-plane methods, and often reduces execution time. Despite their widespread success in solving practical problems of small and intermediate sizes, MIP problems are NP-hard in the worst case. Therefore, given the generic exponential runtime scaling and ever-increasing problem sizes for real-world PO problems, it is crucial to develop techniques that can improve the scope and scale of problems these solvers can handle.

*PO with near-term quantum computing.* In parallel to the development of classical optimizers, quantum computing has shown promise for providing computational speedups for some optimization problems of relevance to science and industry [9–11], particularly in portfolio optimization problems [12,13]. A variety of quantum algorithms have been proposed, including well-studied quantum heuristics like the quantum approximate optimization algorithm (QAOA) [14,15], which has been executed on quantum hardware for small-scale optimization problems [16–19], including portfolio optimization [20–23], with up to approximately 100 variables.

Another approach to quantum optimization is quantum annealing [3], which solves optimization problems by finding the ground state of a classical Ising Hamiltonian, elevated to the quantum domain by describing a collection of interacting

Published by the American Physical Society under the terms of the [Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by/4.0/) license. Further distribution of this work must maintain attribution to the author(s) and the published article's title, journal citation, and DOI.

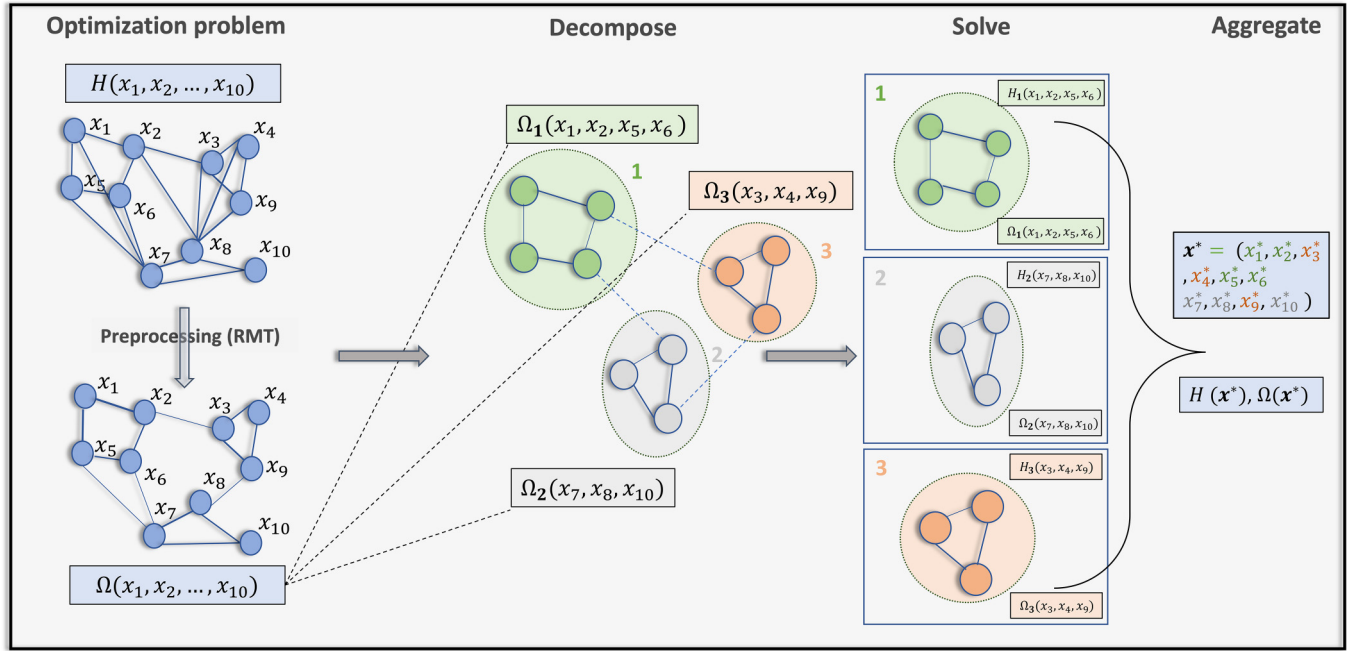


FIG. 1. Diagram of the decomposition pipeline to solve optimization problems, where the objective is denoted by  $H$ , and the constraint set is denoted by  $\Omega$ . Decompose: We first preprocess the correlation matrix using random matrix theory (RMT). The resulting problem is then partitioned into a series of subproblems associated with the objectives  $H_i$  and constraints  $\Omega_i$ ,  $i = 1, 2, 3$ . Solve: Each subproblem is solved (in parallel). Aggregate: The subproblems are aggregated and postprocessed for feasibility checks into the final solution  $\mathbf{x}^*$ . Finally, we check the objective value  $H(\mathbf{x}^*)$ , while ensuring feasibility of the solution vector  $\Omega(\mathbf{x}^*)$ . Note that the aggregation technique shown is simple concatenation, but that one can generalized to more complex techniques, and that potentially different aggregated solutions could be tested.

qubits. Like QAOA, quantum annealing has also been applied to portfolio optimization on quantum hardware [24,25]. There have also been proposals for portfolio optimization based on quantum enhancements of classical algorithms. Specifically, quantum linear systems algorithms [26–28] have been applied to solving reformulated constrained PO problems in both theory [29] and hardware demonstrations [30]. Quantum interior-point methods have also been proposed for this class of optimization problems [31,32]. Finally, although not specifically applied to portfolio optimization to date, quantum algorithms based on quantum walks [33,34] have been shown to improve branch-and-cut algorithms.

A common challenge for implementing quantum algorithms on current hardware is the limited number of qubits, typically ranging from tens to hundreds on near-term devices, depending on the architecture and provider. Consequently, hardware demonstrations to date have been restricted to instances of portfolio optimization problems with a much smaller number of assets than problems solved in production, which can have thousands of variables.

Many financial assets exhibit a specific structure in which groups of assets are correlated within their communities, while being mutually anticorrelated with other communities [35]. Moreover, it has been shown that this structure originates from the real market [36,37]. These results suggest that there are opportunities for improving the performance of classical and quantum solvers by breaking down the problem into smaller subproblems based on such communities. However, to achieve this, it is not sufficient to simply apply standard community detection techniques that use correlation matrices instead of network data, as shown in Ref. [35].

*Our contribution.* In view of the aforementioned challenges for optimization faced by both classical and quantum computers, as well as the evidence of structure present in real-world PO problems, we propose a decomposition pipeline specifically designed to leverage this structure for PO problems, which can be generalized beyond portfolio optimization. Drawing from techniques used for community detection, our method consists of partitioning the original constrained optimization problem into subproblems of constrained optimization, which are each solved separately. The solutions to the subproblems are subsequently aggregated to output a solution to the original problem.

The proposed decomposition pipeline consists of four steps, as shown in Fig. 1. First, we reinterpret the optimization problem by introducing a correlation graph whose nodes correspond to assets, and whose edge weights correspond to the normalized covariance between returns of the assets. We apply preprocessing techniques based on random matrix theory to this graph in order to extract relevant signals. Second, we perform the decomposition by employing a modified spectral clustering based on Newman’s method to partition the graph. The partition corresponds to segmenting the problem into smaller subproblems. Similarly, we decompose the constraint set of the original optimization problem, and we use risk rebalancing techniques before optimizing each subproblem. Third, we solve each of the subproblems individually. Finally, we aggregate the solutions of the subproblems to obtain an approximate solution to the main problem.

Our proposed technique offers two main benefits. First, our decomposition pipeline improves the execution time of classical state-of-the-art solvers on portfolio optimization problems.

We show that, when utilizing the proposed decomposition pipeline and solving each subproblem with a state-of-the-art branch-and-bound based solver, the time-to-solution is significantly reduced (by at least  $3\times$  for the largest problems considered in our numerical experiments, with 1500 variables) as compared to directly solving the problem with the same solver. At the same time, for problems of sufficient size, even the decomposed subproblems become difficult to solve with classical techniques, but are good targets for *quantum* optimization. As such, the second benefit of our technique is that our pipeline contributes to the solution of portfolio optimization problems using quantum computers. Specifically, by leveraging the structure present in the data defining PO problems, we may potentially reduce the number of qubits required to implement the quantum optimization algorithm on quantum devices. We propose to utilize the structure present in typical problem instances to effectively reduce the problem size, thereby making large-scale problems compatible with near-term quantum hardware. Thus, our approach paves the way to near-term hardware demonstrations for practically relevant applications at scale.

It is worth mentioning that decomposition techniques have been applied to a broad range of optimization problems [38,39]. A widely known partitioning procedure for solving MIP problems is Benders' decomposition (BD) [40], wherein the problem is decomposed into small subproblems containing only integer variables and other small linear programs containing only continuous variables. With continuous variables, graph theoretic decomposition algorithms have been explored in Refs. [41,42]. In the context of near-term quantum computing, problem decomposition has been used to tackle graph clustering and graph partitioning problems on small quantum devices [43–46].

## II. PROBLEM STATEMENT AND MOTIVATION

We focus on PO problems in the form of mixed integer quadratically constrained quadratic programming (MIQCQP) problems. The generic form of MIQCQP is given by

$$\begin{aligned} \min_{\mathbf{x} \in \{0,1\}^n} \quad & \mathbf{x}^T A_0 \mathbf{x} + 2\mathbf{b}_0^T \mathbf{x} + \kappa_0 \\ \text{subject to} \quad & \mathbf{x}^T A_i \mathbf{x} + 2\mathbf{b}_i^T \mathbf{x} + \kappa_i \leq 0, \quad \text{for } i = 1, 2, \dots \end{aligned} \quad (1)$$

We use the standard notation of bold lowercase letters, such as  $\mathbf{x}$ , for column vectors, uppercase letters, such as  $A_i$ , for matrices, and lowercase letters, such as  $\kappa_i$ , for scalars. In the above formulation, we write  $\mathbf{x}$  as binary vectors, but it is straightforward to consider any vector  $\mathbf{x}$  whose elements  $x_i$  are integers within a predetermined range of values by, e.g., performing a binary expansion of  $x_i$  and potentially adding new variables and constraints as needed. In the following, we consider two types of PO problems that can be cast as MIQCQPs.

The first PO problem we consider is the canonical Markowitz mean-variance problem with constraints represented using integer variables. Given a universe of  $n \in \mathbb{Z}$  assets to choose from, the objective is to maximize the return while minimizing the exposure to risk. We denote the vector of expected returns as  $\boldsymbol{\mu} \in \mathbb{R}^n$ , the vector of decision variables

with discrete (binary or integer) elements as  $\mathbf{x} \in \mathbb{Z}^n$ , and the covariance matrix of returns as  $\Sigma \in \mathbb{R}^{n \times n}$ . For brevity, we will assume *daily* returns without loss of generality, but it is straightforward to use other types of returns. The daily (rate of) return of an asset is defined as the difference between the closing price and the opening price, divided by the opening price. A standard formulation of this problem is defined by the objective function

$$H(\mathbf{x}) := q \mathbf{x}^T \Sigma \mathbf{x} - \boldsymbol{\mu}^T \mathbf{x}, \quad (2)$$

where the first term in the objective represents the exposure of risk, the second term represents the expected return, and their tradeoff is governed by the risk aversion coefficient  $q \in \mathbb{R}^+$ . The portfolio optimization task is to find a portfolio that minimizes the objective function, i.e.,  $\text{argmin}_{\mathbf{x} \in \{0,1\}^n} H(\mathbf{x})$ .

In practice, PO contains constraints, such as cardinality constraints on lower and upper limits of chosen assets. One can alternatively impose an equality constraint that fixes the exact number of chosen assets to be  $\lceil dn \rceil$  for  $0 \leq d \leq 1$ , which can be expressed as  $\mathbf{1}^T \mathbf{x} = dn$ , where  $\mathbf{1}$  is the all-ones vector.

In this work, we fix the number of assets with a linear equality constraint as

$$\begin{aligned} \min_{\mathbf{x} \in \{0,1\}^n} \quad & q \mathbf{x}^T \Sigma \mathbf{x} - \boldsymbol{\mu}^T \mathbf{x} \\ \text{subject to} \quad & \mathbf{1}^T \mathbf{x} = dn, \end{aligned} \quad (3)$$

where we omit the rounding of  $dn$ . The equality constraint  $\mathbf{1}^T \mathbf{x} = dn$  is equal to two inequality constraints in Eq. (1):  $\mathbf{1}^T \mathbf{x} - dn \leq 0$  and  $-\mathbf{1}^T \mathbf{x} + dn \leq 0$ .

In the second PO problem we consider, one is given a (baseline) portfolio  $\mathbf{x}_b$ , and the goal is to identify a new portfolio  $\mathbf{x}$  that is not far from  $\mathbf{x}_b$ , but which has a smaller overall risk exposure. Namely, the risk of changing  $\mathbf{x}_b$  to  $\mathbf{x}$  as measured by  $\Sigma$  should be minimized. Typically,  $\mathbf{x}_b$  represents a current portfolio that needs to be updated, or it could be the one based on a suggestion of subject matter experts. In this setting, the optimization problem can be formulated as

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{Z}^n} \quad & (\mathbf{x} - \mathbf{x}_b)^T \Sigma (\mathbf{x} - \mathbf{x}_b) \\ \text{subject to} \quad & \mathbf{x}^T \Sigma \mathbf{x} \leq a, \end{aligned} \quad (4)$$

where  $a$  is a scalar strictly less than  $\mathbf{x}_b^T \Sigma \mathbf{x}_b$  so that the optimal solution, if it exists, is different from  $\mathbf{x}_b$ . Notice that  $\mathbf{x}$  is a vector of integers instead of a vector of binaries, but it is straightforward to derive a corresponding MIQCQP as shown in Appendix A.

A popular method to compute the exact solution to the cardinality-constrained PO (3) is the branch-and-bound method [4], which is a recursive method that explores solutions by first *branching* into the subspace of the solutions of smaller subproblems, then obtaining good upper and lower *bounds* that are used to skip searches on a subset of branches. For example, in Eq. (3) the branching can be done by splitting the problem into subproblems with  $x_n$  fixed to either 0 or 1, i.e.,  $H_0(\mathbf{x})$  with  $x_n = 0$  and  $H_1(\mathbf{x})$  with  $x_n = 1$ . Each of the resulting subproblems has one less variable and can also be recursively branched. During the exploration of  $H_0(\mathbf{x})$ , one can obtain a feasible solution (upper bound), and an estimated

optimal solution (lower bound) that may allow one to skip exploring the branches of  $H_1(\mathbf{x})$ . The *integrality gap* is defined as the difference between the lower bound, which is achievable by relaxation, and the upper bound, which is realized by integer variables. Notice that the branching can be done in arbitrary order of the subsets of decision variables, which can greatly influence the time to find optimal solutions (or, solutions with the lowest integrality gaps).

As mentioned in the Introduction, the correlations between financial assets give rise to a clear structure. We provide empirical evidence of this underlying structure originating from the real market [36,37] that the solver leverages to find the solution more efficiently. To illustrate this, we solve the PO problem introduced in Eq. (3) with a linear constraint, constructed using both (i) market data from the S&P 500 and (ii) randomly generated covariance matrices (drawn from the Wishart distribution [47]), which we discuss in Appendix E, and where we compare the time to solution (TTS) of solving these two cases.

There are some PO problems that are especially hard empirically, potentially due to complexity in the objective or constraints. As we will show, in Sec. IV for example, the risk minimization with quadratic constraint [Eq. (4)] formulated with market data (S&P 500) presents a TTS higher than the simpler problem with linear constraint presented before. However, the distinction between random covariance and S&P 500 data is less pronounced, suggesting that classical solvers may struggle to utilize the underlying structure in problems with more complex constraints, further motivating the need for our decomposition framework.

### III. DECOMPOSITION PIPELINE

In this section we describe the proposed decomposition pipeline that leverages the structure of the covariance matrix for typical portfolio optimization problems. Our approach consists of three steps presented with details in the next subsections: preprocessing of the correlation matrix, clustering that utilizes a correlation-based modularity metric, and the partitioning of the constraints, as shown in Fig. 1. As expected, the quality of the solution from the decomposed pipeline depends on the number of clusters and the structure of the decomposed covariance matrices. We give an analytical bound on the expected decrease of the solution of the decomposed pipeline in Appendix B.

#### A. Preprocessing of correlation matrices

In general, given time-series data, as is often the case for the returns of financial assets, the correlation matrices are constructed in the following way. Given the daily returns of  $n$  different assets over a period of  $\mathcal{T}$  days, we define the matrix  $X_{n \times \mathcal{T}} \in \mathbb{R}^{n \times \mathcal{T}}$ , where the  $i$ th row corresponding to the  $i$ th asset is a vector containing the daily returns of that asset across the period of  $\mathcal{T}$  days.

In the setting of finite  $n$  and  $\mathcal{T} \rightarrow \infty$ , given the  $X_{n \times \mathcal{T}}$ , the usual sample covariance estimator is  $\Sigma = (1/\mathcal{T})XX^T$ , and it converges almost surely to the true covariance  $\bar{\Sigma}$ , making it a strongly consistent estimator of the population covariance matrix [48]. In particular, the expected value of the Frobenius

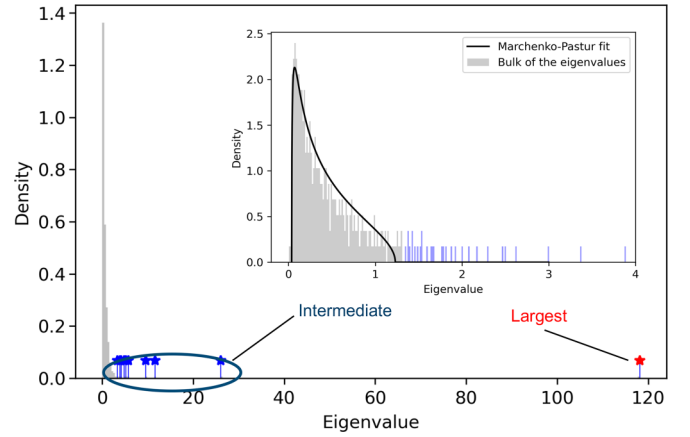


FIG. 2. Eigenvalue density of a correlation matrix derived from S&P 500 market data, with  $n = 484$  stocks and returns data collected from 2014 to 2019 for  $\mathcal{T} = 1260$ , followed by data cleaning. The bulk of the eigenvalues form a dense cluster on the left side of the figure, consistent with the hypothesis of a random correlation matrix, followed by intermediate eigenvalues (shown in blue) and the largest eigenvalue (red star). In the inset, the bulk region is fitted with a Marchenko-Pastur distribution, represented by a smooth black line. The black line fit [59] has a lower edge at  $\lambda_- = 0.09$  and an upper edge at  $\lambda_+ = 1.23$ . The upper edge of the Marchenko-Pastur density can serve as a threshold to identify the noisy part of the correlation matrix.

norm of the difference  $\|\Sigma - \bar{\Sigma}\|_F^2$ , i.e., the mean squared error (MSE), tends to zero in this limit. In practice,  $\mathcal{T}$  is finite. One way of improving the maximum likelihood estimator is with shrinkage, as originally observed in Refs. [49,50]. Another challenge is that in many practical settings, the maximum likelihood estimator is not the right choice because it comes with considerable noise in estimation.

When faced with the challenge of estimating covariance matrices in such regimes, two strategies come to the fore: thresholding and shrinkage estimators. Thresholding serves to simplify the covariance matrix by setting small correlations, those considered to be noise, to zero, which can be beneficial in terms of computational efficiency and interpretability, but can discard potentially valuable information. On the other hand, shrinkage estimators [51–55] aim to improve the estimation of a covariance matrix by combining the sample covariance matrix with a structured estimator, such as the identity matrix.

Once  $\Sigma$  is estimated, preprocessing techniques are applied. Given an estimated  $\Sigma$ , the correlation matrix  $C$  can be written directly as

$$C = D^{-1/2} \Sigma D^{-1/2}, \quad (5)$$

where  $D$  is a diagonal matrix  $D \equiv \text{Diag}(\Sigma_{11}, \Sigma_{22}, \dots, \Sigma_{nn})$ . Thus, each element of  $C$  is given by

$$C_{ij} = \frac{\Sigma_{ij}}{\sqrt{\Sigma_{ii} \cdot \Sigma_{jj}}}. \quad (6)$$

As shown in Fig. 2, for a correlation matrix  $C$  derived from  $n$  completely random time series of duration  $\mathcal{T}$ , a component of the eigenvalue distribution follows the Marchenko-Pastur

(MP) distribution [56], as  $n$  and  $\mathcal{T}$  are both large but with a fixed ratio  $\beta = n/\mathcal{T}$ . The density function of this distribution is given by

$$\rho(\lambda) = \begin{cases} \frac{\sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}}{2\pi\lambda\beta\sigma^2} & \text{if } \lambda_- \leq \lambda \leq \lambda_+, \\ 0 & \text{otherwise,} \end{cases}$$

where  $\lambda_{\pm} = \sigma^2[1 \pm \sqrt{\beta}]^2$  represent the maximum and minimum eigenvalues respectively. For true correlation matrices,  $\sigma = 1$ . However, in practice, we can treat  $\sigma$  to be an adjustable parameter [48,57]. This is because several eigenvalues remain above  $\lambda_+$  and carry some information, thereby decreasing the variance of the effectively random component of the correlation matrix. Similarly,  $\beta$  should be treated as an adjustable parameter [58] because effectively correlated samples are redundant and the sample correlation matrix should behave as if we had observed not  $\mathcal{T}$  samples but an effective number  $\mathcal{T}^*$  or written directly  $\beta^* = (n/\mathcal{T}^*)$ . After the appropriate fits, we obtain  $\lambda_+$  and  $\lambda_-$  as shown in Fig. 2. The utility of the Marchenko-Pastur distribution lies in its ability to characterize those eigenvalues that are predominantly noise-induced. Specifically, eigenvalues within the interval  $[\lambda_-, \lambda_+]$  are attributed to random fluctuations, whereas those exceeding  $\lambda_+$  signify significant underlying structures, often reflecting meaningful relationships in the data [58]. These relevant eigenvalues are highlighted with the blue and red stars in Fig. 2.

To apply this insight to empirical correlation matrices, we decompose the matrix  $C$  into  $C = C_{\text{noise}} + C_{\text{signal}}$ , where

$$C_{\text{noise}} = \sum_{i:\lambda_i \leq \lambda_+} \lambda_i \mathbf{v}_i \mathbf{v}_i^T, \quad (7)$$

$$C_{\text{signal}} = \sum_{i:\lambda_i > \lambda_+} \lambda_i \mathbf{v}_i \mathbf{v}_i^T, \quad (8)$$

the eigendecomposition of  $C$  whose eigenvalues are at most  $\lambda_+$  captures the random components associated with eigenvalues less than or equal to  $\lambda_+$ , and  $C_{\text{signal}}$  represents the structured components reflecting eigenvalues greater than  $\lambda_+$ . The largest eigenvalue,  $\lambda_1$ , leads to top eigenvalue bias in the study of spiked covariance models [58,60].  $\lambda_1$  is typically much greater than  $\lambda_+$ , often corresponding to the global or *market mode* [35,58], a common factor affecting all entities within the dataset. To discern more nuanced relationships, it is essential to further decompose the signal as

$$C_{\text{signal}} = C^* + C_{\text{global}}, \quad (9)$$

where  $C_{\text{global}} = \lambda_1 \mathbf{v}_1 \mathbf{v}_1^T$  represents the market mode, and the relevant structure present in the data is represented by  $C^*$ , which captures correlations within sub-groups of stocks or variables that are influenced similarly by shared factors. Specifically,

$$C^* = \sum_{\substack{i:\lambda_i > \lambda_+ \\ i \neq 1}} \lambda_i \mathbf{v}_i \mathbf{v}_i^T.$$

We leverage  $C^*$  in order to identify communities of assets to define the subproblems as we will present in the next subsections.

## B. Clustering with correlation-based modularity

Our goal is to leverage the structure in the correlation matrix to identify subsets of assets defining the subproblems. To this end, we create the *correlation graph*,  $G$ , whose nodes correspond to assets, and whose edges are weighted by the total correlation between their corresponding nodes (assets in  $C$ ). We then perform clustering to identify communities. Standard community detection algorithms for networks use a modularity metric usually defined in terms of the difference between the actual edge weight  $C_{ij}$  and the expected edge weight under some null model. The expected edge weight is typically estimated based on the product of the degrees  $k_i$  and  $k_j$  of the nodes  $i$  and  $j$ . Thus, the naive correlation based modularity  $Q$  under the community grouping  $\mathbf{g}$  can be expressed as

$$Q(\mathbf{g}) = \frac{1}{\gamma} \sum_{ij} \left[ C_{ij} - \frac{k_i k_j}{\gamma} \right] \delta(g_i, g_j), \quad (10)$$

where  $C_{ij}$  represents the actual edge weight between nodes  $i$  and  $j$ ,  $\gamma$  is the sum of all of the edge weights in the network, and  $g_i$  is the component of  $\mathbf{g}$  which denotes the group assignment for the  $i$ th node.  $\delta(g_i, g_j)$  is the Kronecker delta function equal to 1 if nodes  $i$  and  $j$  are in the same community and 0 otherwise. However such standard community detection algorithms, including the ones developed by Newman [61], do not consider the specific properties of the modularity matrix when derived from correlation matrices [35]. The authors also highlight the need for a revised null model that acknowledges the structure of correlations in the network. A suitable null model for networks represented by correlation matrices must be formulated to account for both the individual properties of the nodes and the overarching network structure. Such a model ensures that the modularity function accurately reflects the network's community division. To this end, we outline modifications to spectral modularity maximization methods for community detection algorithms in Algorithm 1.

We utilize tools from random matrix theory as described in Sec. III A to define a modified correlation matrix  $C^*$  that captures the relevant information. We then use the correlation matrix  $C^*$  to define the edge weight between nodes in the correlation graph. Recall that in Eq. (10) the correlation-based modularity is computed with regards to multiple groups or communities. In this paper, however, we focus on the recursive greedy approach of partitioning into two communities, similarly to Ref. [61]. For this case, let us rewrite  $\mathbf{g}$  as  $\mathbf{z}$  and let  $z_i = +1$  if node  $i$  belongs to a particular community, and  $z_i = -1$  otherwise. Then,  $\delta(g_i, g_j) \equiv (z_i z_j + 1)/2$ , and a modified correlation-based modularity  $Q_c(\mathbf{z})$  can be defined as

$$Q_c(\mathbf{z}) = \frac{1}{\gamma} \sum_{i,j} \frac{C_{ij}^* (z_i z_j + 1)}{2} = \frac{\mathbf{z}^T C^* \mathbf{z}}{2\gamma} + \frac{C_{\text{tot}}^*}{2\gamma}, \quad (11)$$

where  $C_{\text{tot}}^*$  denotes the total edge weight of the cleaned correlation matrix  $C^*$ . By relaxing  $\mathbf{z}$  variables to be real (while satisfying  $\mathbf{z}^T \mathbf{z} = n$  [61]), the vector  $\mathbf{z}$  maximizing  $Q_c(\mathbf{z})$  is easily seen to be the vector matching the signs of the components of the eigenvector of  $C^*$  corresponding to the largest eigenvalue. Additionally, the second term in Eq. (11) can be

## ALGORITHM 1. Modified modularity-based spectral optimization.

**Input:**  $C^*$ : Clean correlation matrix as in Eq. (9)**Output:**  $\mathbf{g}$ : vector of community assignments

```

1: Initialize:  $G \leftarrow C^*$ ,  $U \leftarrow G$ ,  $D \leftarrow \emptyset$ 
2: while  $U \neq \emptyset$  do
3:    $G' \leftarrow \text{pop}(U)$ :
4:    $(\text{isValid}, G_1, G_2) \leftarrow \text{VALID\_SPLIT}(G')$ 
5:   if isValid then
6:      $U \leftarrow U \cup \{G_1, G_2\}$ 
7:   else
8:      $D \leftarrow D \cup \{G'\}$ 
9:   Update  $\mathbf{g}$  to reflect the community labels for the vertices in  $G'$ 
10:  end if
11: end while
12: return  $\mathbf{g}$  as computed from the sets in  $D$ 

```

▷  $G$ : Input graph,  $U$ : Undecided subgraphs

▷ Details in Sec. III B

▷ Check if  $G'$  can be split (details in Algorithm 2)▷ If  $G'$  can be split, store them to be split again

▷ Otherwise, fix the community assignment

dropped from the optimization objective because it does not depend on  $\mathbf{z}$ . In Theorem 1, we show that choosing the eigenvector corresponding to the largest eigenvalue would indeed be optimal for minimizing the drop in solution quality in the setting where we use continuous relaxation of variables.

To further subdivide the communities obtained from the initial bisection, we need to determine the potential modularity change  $\Delta Q_c$ . The graph  $G$  is then split into two new subgraphs ( $G_1$  and  $G_2$ ). Let  $\mathbf{z}$  be a vector representing the vertices in  $G$ . The modularity change for this bisection is denoted by  $\Delta Q_c^{(G_1|G_2)}$ , and is given by

$$\begin{aligned}
\Delta Q_c^{(G_1|G_2)} &= \frac{1}{\gamma} \left[ \sum_{i,j \in G_1} C_{ij}^* + \sum_{i,j \in G_2} C_{ij}^* - \sum_{i,j \in G} C_{ij}^* \right] \\
&= \frac{1}{\gamma} \left[ \sum_{i,j \in G} C_{ij}^* \frac{1 + z_i z_j}{2} - \sum_{i,j \in G} C_{ij}^* \right] \\
&= \frac{1}{2\gamma} \left[ \sum_{i,j \in G} C_{ij}^* z_i z_j - \sum_{i,j \in G} C_{ij}^* \right] \\
&= \frac{\mathbf{z}^T C_G^* \mathbf{z}}{2\gamma} - \frac{\tilde{C}_{GG}^*}{2\gamma},
\end{aligned} \tag{12}$$

where  $C_G^*$  represents the submatrix of  $C^*$  restricted to the subset of nodes within graph  $G$ , and  $\tilde{C}_{GG}^* := \sum_{i,j \in G} C_{ij}^*$ . As with the initial bisection,  $\mathbf{z}$  is chosen to maximize  $\Delta Q_c^{(G_1|G_2)}$  by selecting its elements to match the sign of the eigenvector corresponding to the largest eigenvalue of the matrix  $C_G^*$  [35,62]. Then, the proposed modularity-based spectral method continues iterating this procedure until no further bisection can increase the modularity. We summarize the steps in Algorithm 1 and Algorithm 2 (see Appendix C).

The proposed method differs from standard spectral clustering in that it specifically aims to maximize a modularity function that measures the strength of division of a network into modules or communities. This is achieved by eigendecomposing the modularity matrix, a matrix that represents the difference between the actual adjacency matrix of the network and a null model. The method involves recursively or iteratively bisecting the network based on the leading eigenvector

of the modularity matrix, contrasting with spectral clustering, which typically partitions a network based on the eigenvectors of the Laplacian matrix and does not necessarily aim to maximize modularity.

Another alternative for a modified modularity metric is to utilize a *normalized*  $C^*$  instead of  $C$ , which is defined as

$$(\tilde{C}^*)_{ij} \leftarrow C_{ij}^* - \frac{k_i k_j}{\gamma}, \tag{13}$$

where  $k_i = \sum_j C_{ij}^*$ . Note that  $\mathbf{1} = (1, 1, \dots)^T$  is a trivial eigenvector of the normalized  $\tilde{C}^*$  with eigenvalue 0, because the row sums and the column sums of  $\tilde{C}^*$  are 0. This is reminiscent of a property of the matrix known as the graph Laplacian [63]. Hence, the optimal eigenvector of  $\tilde{C}^*$  is orthogonal to the all  $\mathbf{1}$  vector [61], thus satisfying  $\sum_i z_i = 0$ . This ensures that we can find a bipartition corresponding to the positive and negative components of the optimal eigenvector of the modified modularity matrix. We can use this observation further to perform the clustering process in order to find all communities smaller than a given threshold. We introduce Algorithm 3 (which is a slight modification of Algorithm 1, see Appendix C) that iteratively decomposes the problem further until the number of variables in each problem is less than a certain threshold. Specifically, at each step, if a community size is over a threshold, we rerun the community detection algorithm as many times needed until all the subcommunities generated are within the expected size. Note that we can improve this algorithm by regrouping small communities together, which should reduce the solution error to the original problem.

When executing our algorithms, we need to ensure that each node is assigned to a community, or that each node belongs to a community whose size is at most the maximum allowed size for a community (denoted by  $\phi$ ) after executing Algorithm 1 or Algorithm 3, respectively. For this, we introduce the *pop* method to choose a subgraph whose nodes have not been assigned to a community in Algorithm 1 or to choose a subgraph whose size exceeds the threshold in Algorithm 3. We denote the set of subgraphs that have not yet been assigned to a community by  $U$ . There are several possible strategies to pick a subgraph from  $U$ : (i) pick the first element in the first-in-first-out order, (ii) pick a subgraph uniformly at random,

(iii) choose it with probability proportional to its size, or (iv) prioritize the largest subgraph. In practice, we observe that approaches (i) and (iii) outperform the other choices.

### C. Partitioning of constraints

In the previous subsection, we have shown how to divide the initial asset universe of the problem into subsets of assets to define subproblems. To do this, it is also required to partition the constraints of the initial problem. We show how to do this for two types of constraints in next subsections.

#### 1. Cardinality constraints

Proceeding with the earlier formulation of dividing the asset universe into  $K$  communities, we denote the communities by the index  $k$ . Each community has  $n_k$  assets, expected returns  $\mu_k$ , and covariance matrix  $\Sigma_k$ . Within each community  $k$ , the subproblem is formulated as

$$\min_{\mathbf{x}_k \in \mathbb{Z}^{n_k}} H_k(\mathbf{x}) : -\mu_k^T \mathbf{x}_k + q' \mathbf{x}_k^T \Sigma_k \mathbf{x}_k, \quad (14)$$

where  $\mathbf{x}_k$  denotes the vector of asset weights in community  $k$  with  $n_k$  assets. Each community adheres to a local cardinality constraint  $\Omega_k : \mathbf{1}^T \mathbf{x}_k = dn_k$ . After solving these subproblems, the solutions  $\mathbf{x}_1^*, \mathbf{x}_2^*, \dots, \mathbf{x}_K^*$  are recombined to respect the global cardinality constraint:  $\Omega : \sum_{k=1}^K \mathbf{1}^T \mathbf{x}_k^* = dn$ . The validity of this combined solution is determined by its adherence to both local community constraints and the global constraint. In order to satisfy the global constraint  $\Omega$ , we can apply rounding. For the first  $K-1$  communities (i.e.,  $k = 1, 2, \dots, K-1$ ), the number of selected assets is the one output by each separate subproblem. Then, the number of assets for the  $K$ th community is enforced to be  $\lfloor dn \rfloor - \sum_{k=1}^{K-1} \lfloor dn_k \rfloor$ .

Similarly, we restrict the risk of the subproblems relative to the total exposure of risk of the initial problem. To this end, we perform risk rebalancing, which adjusts the portfolio to maintain a desired level of risk exposure. We modify the objective function in Eq. (14) by replacing  $q$  with  $q'$  as

$$q' = q \times \left( \frac{\sum_k \|\mu_k\|_2 / \|\mu\|_2}{\sum_k \|\Sigma_k\|_F / \|\Sigma\|_F} \right). \quad (15)$$

To motivate the need for rebalancing, observe that the risk factor  $q$  is sensitive to the units in which returns and variances are measured. For example, if we scale all prices by a constant factor  $\alpha$ , then the risk factor  $q$  must be scaled by  $1/\alpha$  to retain the same solution. For a given risk tolerance, therefore, the corresponding  $q$  is dependent on the norms of  $\Sigma$  and  $\mu$ . After community detection, we are effectively solving portfolio optimization with a block diagonal covariance matrix with blocks given by  $\Sigma_k$ . The proposed rebalancing attempts to preserve the risk tolerance under this modification. We cannot analytically guarantee this, because the subproblems are MIQPs and are subject to rounding. However, this rule is empirically observed to be very effective in minimizing the loss in solution quality compared to the original problem.

#### 2. Quadratic constraints

Consider the particular PO problem with a quadratic constraint introduced in Eq. (4). In a decomposed optimization framework, the overall problem is partitioned into a series

of subproblems. Each subproblem  $k$  is associated with an objective involving a quadratic term,

$$(\mathbf{x}_k - (\mathbf{x}_b)_k)^T \Sigma_k (\mathbf{x}_k - (\mathbf{x}_b)_k),$$

where  $\mathbf{x}_k$  represents the decision variable for the  $k$ th subproblem, and  $\Sigma_k$  is a symmetric positive definite matrix associated with the  $k$ th subproblem. The subproblems are constrained by  $\mathbf{x}_k^T \Sigma_k \mathbf{x}_k \leq w_k a$ , where  $w_k$  represents the weight on the  $k$ th subproblem,  $(\mathbf{x}_b)_k$  represents the elements of  $\mathbf{x}_b$  vector corresponding to the community  $k$ , and  $a$  is a scalar parameter shared across all subproblems. One simple choice for  $w_k$  can be to use the ratio of nodes present in the  $k$ th subproblem over the total number of assets.

Satisfying these constraints locally does not guarantee that the global constraint is satisfied. To enforce the feasibility of the global constraint, we introduce a new matrix  $\Sigma'' = s \Sigma'$ , where  $\Sigma'$  is a block diagonal matrix constructed from the matrices  $\Sigma_k$ , and  $s$  is a scalar. This matrix  $\Sigma''$  is designed to be diagonally dominant over the original covariance matrix  $\Sigma$ , thereby ensuring that  $\Sigma'' - \Sigma \geq 0$ . To ensure this, we need to derive conditions on the scalar  $s$ . The criterion is derived based on the relationship

$$s \geq \frac{\lambda_{\max}(\Sigma)}{\lambda_{\min}(\Sigma')} \geq 1, \quad (16)$$

where  $\lambda_{\max}(\cdot)$  and  $\lambda_{\min}(\cdot)$  represent the maximum and the minimum eigenvalues, respectively. The minimum eigenvalue of  $\Sigma'$  is the minimum among all the block diagonal constituents, because their union forms the eigenset of  $\Sigma'$ . This condition ensures that the scaling factor  $s$  sufficiently elevates the eigenvalue spectrum of  $\Sigma'$  to meet or exceed that of  $\Sigma$ . With the modified constraint for each subproblem being

$$\mathbf{x}_k^T \Sigma_k \mathbf{x}_k \leq \frac{w_k a}{s},$$

and assuming the normalization condition  $\sum_k w_k = 1$  is satisfied for feasible solutions, the aggregate of these constraints across all subproblems implies

$$\mathbf{x}^T \Sigma' \mathbf{x} = \sum_k \mathbf{x}_k^T \Sigma_k \mathbf{x}_k \leq \frac{a}{s} \sum_k w_k \leq a.$$

Alternatively, the initial step in this strategy involves setting the weight of each subproblem's constraint,  $w_k a$ , to be proportional to a predefined quadratic form:  $w_k a = r \cdot (\mathbf{x}_b)_k^T \Sigma_k (\mathbf{x}_b)_k$ , where  $r < 1$  is a constant. The choice of  $r$  is explained in Appendix D so that

$$a = r \sum_k (\mathbf{x}_b)_k^T \Sigma_k (\mathbf{x}_b)_k < \mathbf{x}_b^T \Sigma \mathbf{x}_b \quad (17)$$

eliminates the trivial solution  $\mathbf{x}_b$  in Eq. (4). The suppression factor  $s$  for feasibility of the constraints can be further introduced modifying the initial weighting to  $w_k a/s = (r/s) \cdot (\mathbf{x}_b)_k^T \Sigma_k (\mathbf{x}_b)_k$ , where increasing  $s$  serves to systematically tighten the constraints. The choice of  $s$  is critical: it must be large enough to ensure the feasibility of all subproblems while avoiding overly restrictive constraints that might preclude feasible solutions. In practice, we do a simple search over a range of values starting from 1.

#### IV. NUMERICAL RESULTS

We now numerically assess the performance of our decomposition pipeline in its application to PO problems based on real-world data.

*Problem instances and figures of merit.* The dataset utilized in this work is based on the Russell 3000 index, with daily returns collected over a period of 1000 days starting from January 2010. The instances range in size from 90 to 1500 assets. We utilize ten seeds, each of which corresponds to incrementally adding a random pool of assets in step sizes. Branch and bound (B&B) solvers are characterized by the MIP gap (default  $10^{-4}$ ), which corresponds to the difference between the upper and lower bounds on the solution:

$$\text{MIP gap} = \frac{|H_B - H_I|}{|H_I|}.$$

Here,  $H_I$  is the best known integral bound on the objective value (i.e., the best integer feasible solution found so far, also referred to as incumbent), and  $H_B$  is the best known upper bound, being this the optimal value corresponding to the relaxed problem in the B&B procedure. The solver terminates successfully as soon as it has proven that the gap of the best found solution is below a specified small value; we refer to this time as time to solution (TTS).

This time is one figure of merit, together with the solution quality  $H$ . The problems considered for the benchmark are the two PO problems introduced in Sec. II. We benchmark the performance of the proposed decomposition, where we solve each subproblem utilizing a B&B solver. We compare to the results obtained through *direct optimization*, which refers to the optimization of the full problem. For the problem with cardinality constraints, we utilize the default MIP gap of  $10^{-4}$  for both direct optimization and the subproblems in the decomposition pipeline. The reason is that, for a MIP gap of this order, both *direct optimization* and the decomposition pipeline achieve competitive solution quality  $H$  as a function of the problem size for both optimization problems considered [see Figs. 3(a) and 5(a)]. Therefore, one can fairly compare the TTS for the two methods for this value of the MIP gap. In the inset plot of Fig. 3(a), we empirically observe that the drop in solution quality is within  $\sim 5\%$  for larger system sizes. Hence, for completeness, we also compare the decomposition pipeline against direct optimization using a larger MIP gap ( $5 \times 10^{-2}$ ) and show improvements in run times. In the inset plot of Fig. 3(a) we can see that indeed, the loss in solution quality as a consequence of the approximations imposed by the pipeline is bounded due to the structure present in the data. The TTS for direct optimization can be reduced by widening the MIP gap at the cost of a reduced solution quality, denoted by  $H$ . However, we observed that even with an increased MIP gap (0.1), the direct optimization method still requires a longer TTS while only achieving a solution quality comparable to that of the decomposition pipeline. Detailed results are provided in Appendix E.

Moreover, for the decomposition pipeline we benchmark the performance without restricting the community sizes, part of the decomposition component of the pipeline, and by restricting the size of the communities up to 30. The motivation of this number is to make these subproblems compatible with

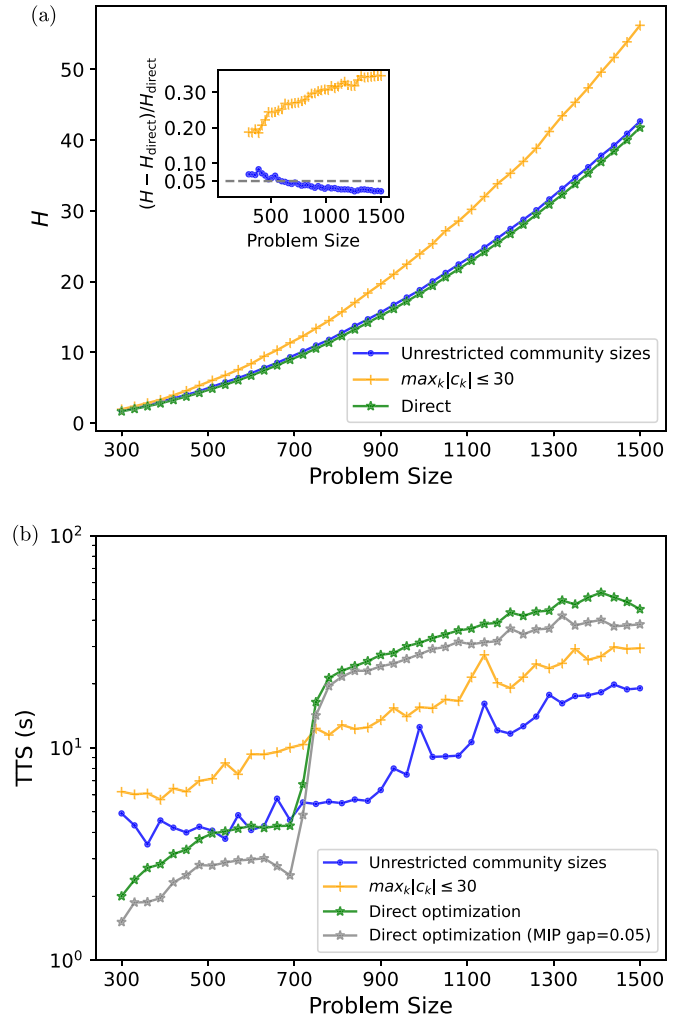


FIG. 3. Performance as a function of the problem size of the PO with linear cardinality constraint set at  $n/2$  [Eq. (3)]. The points correspond to the mean values over ten seeds for each problem size. Panel (a) (top) shows the solution quality  $H$  and panel (b) (bottom) the time to solution (TTS). The line (stars) corresponds to directly optimizing with the green line corresponding to the upper bound of the MIP gap set at the default ( $10^{-4}$ ). The blue line (dots) and the orange (plus signs) refer to the decomposition pipeline, where the first one corresponds to the case of unrestricted community size and latter to restricting the largest community size to 30. The inset of panel (a) shows the relative drop in objective for our decomposition pipeline with and without restrictions over the community size. The dashed (gray) line marks the drop or gap percentage at 0.05. We also plot the TTS for optimization runs with a corresponding MIP gap of 0.05 in gray. For this MIP gap, the corresponding solution quality partially overlaps with the results of direct optimization with the default MIP gap (green curve) in panel (a) and, as such, we omit this curve from the plot for visual clarity. A discussion on the solution quality for different MIP gaps can be found in Appendix E.

near-term quantum devices that are characterized by a limited number of qubits (usually in the order of tens or hundreds). Given that one generally needs a number of ancilla qubits to faithfully embed the given problem in the underlying quantum hardware, we limit the size of the subproblems (i.e., communities) to 30.

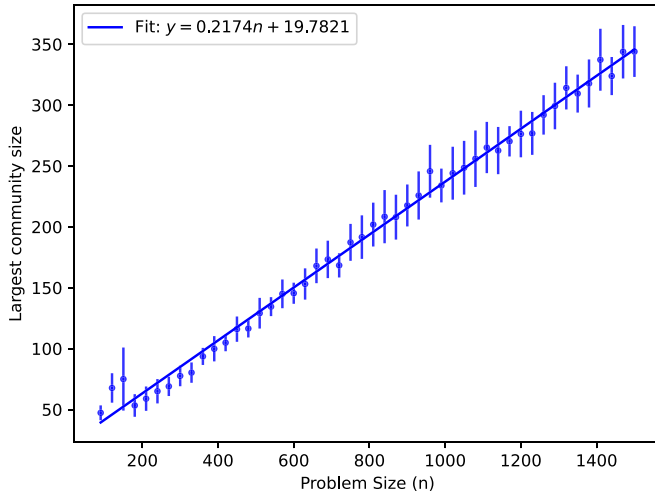


FIG. 4. Size of the largest community against problem size ( $n$ ). The error bars correspond to the standard deviation over ten seeds for each problem size. The median is fit with a linear curve with slope  $\sim 0.21$ .

In our experiments, we obtained results using both Gurobi and CPLEX. Our approach consistently obtained results with a better TTS using Gurobi, and as such we report benchmark results using Gurobi; see Appendix E for a discussion. Note that the TTS reported corresponds to running the optimization of each subproblem sequentially, and each with the default MIP gap ( $10^{-4}$ ). Additionally, the proposed decomposition pipeline allows parallelization of the optimization of each subproblem, improving the end-to-end run time. The classical hardware on which our numerical experiments were run, together with more details, are specified in Appendix E.

The results for the problem considered with linear constraints are shown in Fig. 3. We find that performance within 5% of the optimum is possible while effectively reducing the system size by approximately 80% (see Fig. 4) and saving a factor of  $3\times$  in run time for real-world problems with  $\sim 1500$  assets. For a smaller problem size of 800 assets, the end-to-end pipeline takes approximately 7 s when we employ the community detection algorithm without any restrictions on the community sizes, and approximately 11 s when we add an additional restriction on size of the communities to be less than 30, about  $2\times$  faster than direct optimization without any decomposition, which needs more than 20 s. In Fig. 3(b), we observe that the end-to-end pipeline with restriction on community size (orange plus signs) takes longer than the unrestricted case (blue dots) because the increment in time for finding smaller communities is more than the increment in the combined time taken for the optimization of each subproblem. We observe sudden jumps on direct optimization, which we do not observe on the proposed decomposition pipeline. We hypothesize that the internal mechanics (tooling) used within the black-box MIP solver (e.g., for heuristic preprocessing, and presolving) change abruptly for those problem sizes.

In order to better understand the optimization time of our decomposition pipeline, we plot the size of the largest community found in Fig. 4. We observe an effective reduction of approximately 80% in problem size, indicating that indeed the structure in the data enables us to decompose the problem,

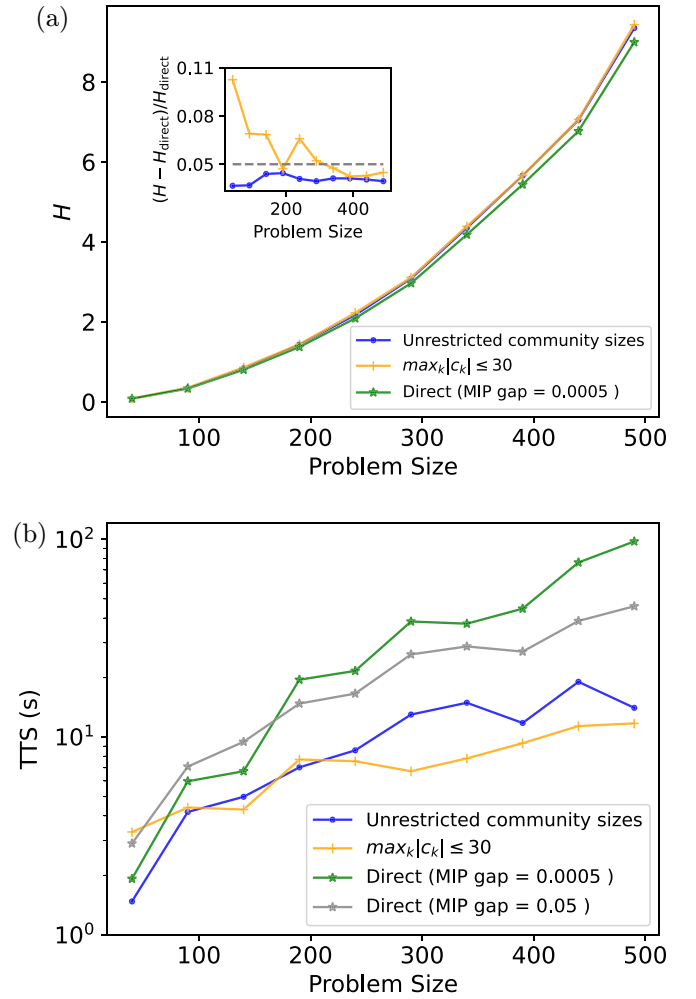


FIG. 5. Performance as a function of the problem size of the optimization of the risk minimization problem with quadratic constraint set at  $a = 0.9 \times \mathbf{x}_b^T \Sigma \mathbf{x}_b$  [Eq. (4)]. The points correspond to the mean values across the ten seeds for each problem size. Panel (a) (top) shows the solution quality  $H$  and panel (b) the time to solution (TTS). The blue line (dots) and the orange (plus signs) refer to the proposed decomposition pipeline, where the former corresponds to the scenario with no restriction on community size, while the latter corresponds to the one where the largest community size is restricted to 30. The lines marked with stars correspond to directly optimizing the portfolio, with the green-colored lines corresponding to MIP gap set at  $5 \times 10^{-4}$  and the gray lines corresponding to a gap of  $5 \times 10^{-2}$ . The solution quality for the larger MIP gap partially overlaps with the results of direct optimization using the default MIP gap (green curve) and, as such, we omit this curve in panel (a) for visual clarity.

further explaining the advantage of using our pipeline over optimizing the full problem instances in time to solution.

For the optimization problem with quadratic constraint shown in Eq. (4) we perform a similar analysis in Fig. 5. Differently from the previously discussed experiments, we set the larger MIP gap to be  $5 \times 10^{-4}$ , because otherwise the TTS with default MIP gap  $10^{-4}$  reaches a timeout of 1000 for problem sizes as small as  $n \approx 160$  (see Fig. 6). We observe that, while the performance in quality of solution of the decomposition pipeline is comparable to the performance of direct optimization, it is significantly faster in terms of TTS.

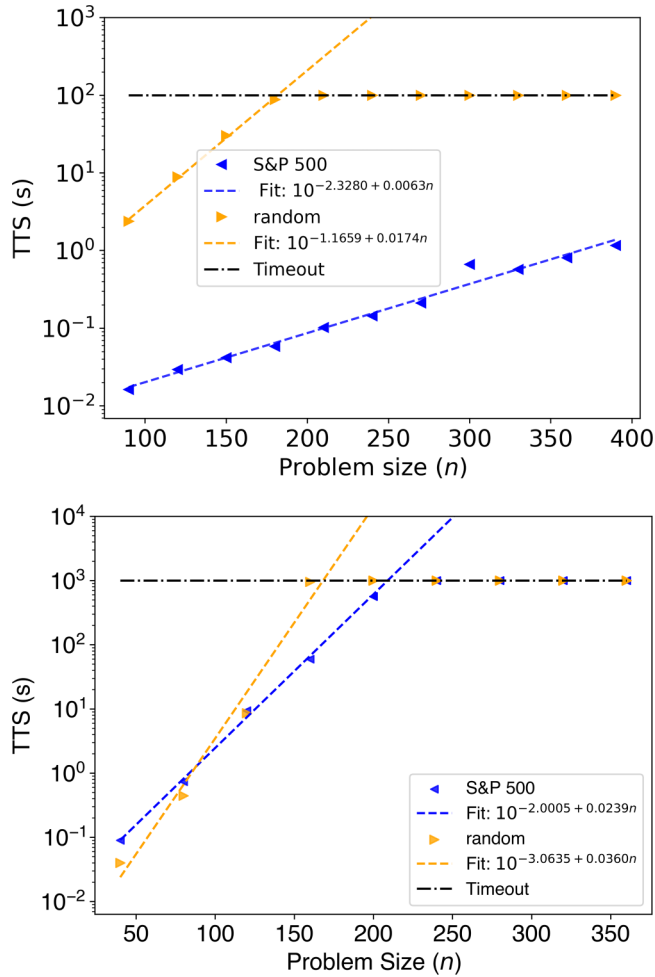


FIG. 6. Median time-to-solution (TTS) over ten seeds as a function of the problem size ( $n$ ) for solving the portfolio-optimization problem with one cardinality constraint [Eq. (3), top panel] and a quadratic objective with a single quadratic constraint, as functions of problem size  $n$  (bottom panel). Top panel: TTS scaling for random covariance (orange right triangles),  $\propto 10^{0.00174n}$ , is worse than for S&P 500 (blue left triangles),  $\propto 10^{0.0063n}$ . The problems with random covariance reach the timeout of 100 s at problem size  $n \sim 200$ , but for the S&P 500 data set we do not hit the timeout. Bottom panel: TTS scaling for random covariance (orange right triangles),  $\propto 10^{0.036n}$ , is worse than for the S&P 500 data set (blue right triangles),  $\propto 10^{0.0239n}$ . The random covariance problem exceeds the 1000 s timeout at  $n > 160$ , while the S&P 500 data set approaches the timeout at  $n > 240$ . The solver used in the experiments is GUROBI.

It is important to note that, without the additional steps discussed in Sec. III C, the objective values obtained through the decomposition pipeline can be lower than the objective values obtained via direct optimization; however, the corresponding aggregated solutions become infeasible. We emphasize that, in our results, we always use the rescaling described in Sec. III C to enforce feasibility.

## V. DISCUSSION

In summary, we have proposed and implemented a modular decomposition pipeline that leverages the community

structure inherent to the underlying data to reduce large-scale PO problems into a family of much smaller (and independent) subproblems. Using real-world data from the Russell 3000 index, we have applied our decomposition logic to two different (NP-hard) PO problems, showing that an approximately 80% reduction in problem size can be achieved while maintaining solution quality within a minimal error. Such a reduction in effective problem size opens the door for hybrid integration with near-term quantum devices, making large-scale PO problems with thousands of assets potentially compatible with quantum hardware with hundreds of qubits. It would be interesting to investigate a potential generalization of our framework towards a larger class of constrained PO problems and beyond in subsequent work.

## VI. OUTLOOK

Our decomposition pipeline offers a promising avenue for leveraging emerging quantum computers to solve large-scale portfolio optimization problems. Quantum hardware is still in its infancy, and limited qubit availability remains a significant challenge. Our pipeline addresses this challenge by breaking down complex portfolio management problems into smaller subproblems, each of which is compatible with near-term quantum devices. In particular, the subproblems generated by our pipeline align well with the capabilities of quantum annealers and the quantum approximate optimization algorithm (QAOA). These quantum algorithms have demonstrated success in solving optimization problems on a small scale [18–20], and their application to the subproblems we generate could provide valuable insights into the practical utility of quantum computing in finance. Recent studies have shown that quantum annealing can effectively tackle portfolio optimization problems by finding the ground state of an Ising Hamiltonian [24,25], while QAOA has been applied to similar problems with promising results [22]. These approaches are particularly relevant to the subproblems derived from our pipeline, which are characterized by reduced complexity and size, making them feasible for current quantum hardware. Moreover, the integration of quantum computing into our framework could pave the way for hybrid quantum-classical optimization strategies. By leveraging the strengths of both classical and quantum solvers, we can potentially achieve superior performance in solving large-scale optimization problems. This hybrid approach could be particularly beneficial in scenarios where classical solvers struggle with scalability or complex constraints [10,11]. Looking ahead, we envision conducting experiments on quantum hardware, such as D-Wave’s quantum annealer, to empirically validate the performance of our pipeline in a quantum setting. Additionally, simulations of quantum annealing and QAOA on GPUs or supercomputers could provide further insights into the scalability and efficiency of these algorithms when applied to our subproblems [64].

In conclusion, our decomposition pipeline not only enhances the applicability of classical optimization techniques, but also opens new pathways for the integration of quantum computing into financial optimization. As quantum technology advances, we anticipate that our approach will contribute

to the development of practical quantum solutions for real-world optimization challenges.

### ACKNOWLEDGMENTS

We thank Brandon Augustino from the Global Technology Applied Research team at JPMorganChase for detailed reviews of the manuscript and Alex Buts and Raj Ganesan, also from Global Technology Applied Research team, for their contributions to the execution of the numerical simulations. We thank Eun Leonard and Grant Chang from JPMorganChase for discussion on the quadratically constrained optimization problems. We thank Victor Bocking and Peter Sceusa for their support.

This paper was prepared for informational purposes with contributions from the Global Technology Applied Research center of JPMorgan Chase & Co. This paper is not a product of the Research Department of JPMorgan Chase & Co. or its affiliates. Neither JPMorgan Chase & Co. nor any of its affiliates makes any explicit or implied representation or warranty and none of them accept any liability in connection with this paper, including, without limitation, with respect to the completeness, accuracy, or reliability of the information contained herein and the potential legal, compliance, tax, or accounting effects thereof. This document is not intended as investment research or investment advice, or as a recommendation, offer, or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction.

### DATA AVAILABILITY

An open source demo version of our code is publicly available at [65].

### APPENDIX A: TRANSFORMING INTEGER PROGRAMMING TO A STANDARD MIQCQP

In Eq. (1), we showed a standard form of MIQCQP with binary decision variables. However, the decision variables in the optimization problem of Eq. (4) are integers. Here, we illustrate how to use a binary expansion to convert the integer decision variables into binaries by adding additional constraints.

Let us assume that each element of  $\mathbf{x}$  is at most  $m \in \mathbb{Z}^+$ , and let  $l$  be a positive integer such that  $2^{l-1} \leq m < 2^l$ . For each  $j$ th element of  $\mathbf{x}$ ,  $x_j$ , we can assign  $l$  binary decision variables  $y_{jl}$  such that

$$x_j = \sum_{k=0}^{l-1} y_{jk} 2^k \quad \text{for } j = 1, \dots, n. \quad (\text{A1})$$

We can then use Eq. (A1) to transform Eq. (4) into Eq. (1). To do so, additional inequalities must be added in order to guarantee that each element of  $\mathbf{x}$  is at most  $m$ , namely

$$\sum_{k=0}^{l-1} y_{jk} 2^k \leq m \quad \text{for } j = 1, \dots, n.$$

The rest of the transformation follows straightforwardly, thus allowing us to solve an integer problem leveraging binary variables.

### APPENDIX B: ANALYTICAL BOUND ON THE SOLUTIONS WITH DECOMPOSED PIPELINES

Here, we discuss the quality of solutions we obtain from the decomposition pipeline to solve the problems as in Eq. (2). In particular, we derive a bound on the degradation in the objective function under the decomposition pipeline. From this bound, we can devise strategies to mitigate the degradation. For example, as in Algorithms 1 and 2, we show that the communities are constructed using the largest eigenvalue and eigenvector, and the number of communities is made to be as small as possible. These facts can be explained directly from the bound given in the following theorem.

*Theorem 1 (Gap between optimal direct and decomposed solutions).* Let  $H(\mathbf{x})$  be the objective function as in Eq. (2) for  $\mathbf{x} \in \mathbb{R}^n$ . Let  $\Sigma$  be the covariance matrix and  $\Sigma'$  be the block-diagonal matrix derived from  $\Sigma$  using the decomposition pipeline. Let  $\mathbf{x}^*$  and  $\mathbf{x}'$  be the optimal assignments for the direct  $\Sigma$  and decomposed  $\Sigma'$ , respectively. Let  $K$  be the number of communities in the pipeline so that  $\Sigma' \equiv \text{Diag}(\Sigma'_{11}, \dots, \Sigma'_{KK})$  and  $\Gamma \equiv \min_k \|\Sigma_{kk}\|$ . Then, it holds that

$$|H(\mathbf{x}^*) - H(\mathbf{x}')| \leq \frac{\|\Sigma^{-1}\| \cdot \|\mu\|^2}{4q} \left( \frac{K \|\Sigma - \Sigma'\|}{\Gamma} \right)^2.$$

*Proof.* For completeness, we rewrite  $H(\mathbf{x})$  as

$$H(\mathbf{x}) \equiv -\mu^T \mathbf{x} + q \mathbf{x}^T \Sigma \mathbf{x} \quad (\text{B1})$$

and  $H'(\mathbf{x})$  as

$$H'(\mathbf{x}) \equiv -\mu^T \mathbf{x} + q \mathbf{x}^T \Sigma' \mathbf{x}. \quad (\text{B2})$$

In the limit of a large time horizon, the covariance matrices  $\Sigma$  and  $\Sigma'$  will both be positive definite, which can be seen from the form of the Marchenko-Pastur distribution, or from the fact that the Cholesky factors have sufficient dimensions to make the covariance matrices full rank. Then, by the convexity of the objective functions, one can see that  $H(\mathbf{x})$  and  $H'(\mathbf{x})$  are optimized, respectively, at

$$\mathbf{x}^* = \frac{1}{2q} \Sigma^{-1} \mu \quad \text{and} \quad \mathbf{x}' = \frac{1}{2q} (\Sigma')^{-1} \mu.$$

Thus, we can derive

$$\begin{aligned} |H(\mathbf{x}^*) - H(\mathbf{x}')| &= \frac{1}{4q} |\mu^T [2(\Sigma'^{-1} - \Sigma^{-1}) + (\Sigma^{-1} - (\Sigma'^{-1})^T \Sigma \Sigma'^{-1})] \mu| \\ &= \frac{1}{4q} |\mu^T (2\Sigma'^{-1} - \Sigma^{-1} - \Sigma'^{-1} \Sigma \Sigma'^{-1}) \mu| \\ &= \frac{1}{4q} |\mu^T (\Sigma'^{-1} - \Sigma^{-1} - \Sigma'^{-1} \Sigma \Sigma'^{-1} + \Sigma'^{-1}) \mu| \\ &= \frac{1}{4q} |\mu^T [\Sigma'^{-1} (\Sigma - \Sigma') \Sigma^{-1} - \Sigma'^{-1} (\Sigma - \Sigma') \Sigma'^{-1}] \mu| \\ &= \frac{1}{4q} |\mu^T [\Sigma'^{-1} (\Sigma - \Sigma') (\Sigma^{-1} - \Sigma'^{-1})] \mu| \end{aligned}$$

## ALGORITHM 2. Valid splitting function.

---

```

1: function VALID_SPLITG
2:   Compute the modularity matrix  $Q_c$  for  $G$  ▷ See Eq. (11)
3:   Compute the highest eigenvalue and eigenvector of  $Q_c$ 
4:    $I_{\text{pos}} = \{i \mid \text{eigvec}[i] \geq 0\}$ ,  $I_{\text{neg}} = \{i \mid \text{eigvec}[i] < 0\}$ 
5:   if ( $I_{\text{pos}} \neq \emptyset$  and  $I_{\text{neg}} \neq \emptyset$ ) then
6:      $G_1, G_2 =$  subgraph of  $G$  with vertices in  $I_{\text{pos}}, I_{\text{neg}}$ 
7:     if  $\Delta Q_c^{(G_1|G_2)} > 0$  (See Eq. (12)) then
8:       return (true,  $G_1, G_2$ ) ▷  $G$  can be split into  $G_1$  and  $G_2$ 
9:     else
10:      return (false,  $G, \emptyset$ ) ▷  $G$  cannot be split
11:    end if
12:  else
13:    return (false,  $G, \emptyset$ )
14:  end if
15: end function

```

---

$$\begin{aligned}
&= \frac{1}{4q} |\boldsymbol{\mu}^T [\Sigma'^{-1}(\Sigma - \Sigma')\Sigma^{-1}(\Sigma' - \Sigma)\Sigma'^{-1}] \boldsymbol{\mu}| \\
&\leq \frac{1}{4q} \|\boldsymbol{\mu}\|^2 \|\Sigma'^{-1}\|^2 \|\Sigma - \Sigma'\|^2 \|\Sigma^{-1}\|,
\end{aligned}$$

where we have used the fact that  $\Sigma$  and  $\Sigma'$  are positive definite to set  $(\Sigma^{-1})^T = \Sigma^{-1}$  and similarly for  $\Sigma'$ .

Finally, the bound in the theorem is obtained by noticing that  $\|(\Sigma')^{-1}\| \leq \sum_k \|((\Sigma')_{kk})^{-1}\| \leq K/\Gamma$ . ■

The bound in the theorem enables us to devise a decomposition strategy to mitigate the degradation of the solutions by choosing as few communities  $K$  as possible and by partitioning the communities so that each  $\|\Sigma'_{kk}\|$  is large and the matrix  $\Sigma'$  is as close to  $\Sigma$  as possible.

### APPENDIX C: SPECTRAL CLUSTERING ALGORITHM WITH THRESHOLD OF COMMUNITY SIZE

We present the splitting function (Algorithm 2) which is used in Algorithm 1. We also discuss Algorithm 3 to find communities smaller than a given threshold on the size of the communities.

### APPENDIX D: RESCALING FACTOR IN OPTIMIZATION PROBLEMS WITH QUADRATIC CONSTRAINTS

We argued that the rescaling factor  $r$  is smaller than 1 in Sec. III C 2 so that  $a \leq \mathbf{x}_b^T \Sigma \mathbf{x}_b$ . Here, we give its explanation based on the continuous setting for the optimization problem (4) described by a convex objective function with a quadratic constraint of the form

$$\begin{aligned}
&\min_{\mathbf{x} \in \mathbb{R}^n} \quad (\mathbf{x} - \mathbf{x}_b)^T \Sigma (\mathbf{x} - \mathbf{x}_b) \\
&\text{subject to} \quad \mathbf{x}^T \Sigma \mathbf{x} \leq a.
\end{aligned} \tag{D1}$$

The Lagrangian  $\mathcal{L}(x, \lambda)$  for this constrained optimization problem is defined as

$$\mathcal{L}(x, \lambda) = (\mathbf{x} - \mathbf{x}_b)^T \Sigma (\mathbf{x} - \mathbf{x}_b) + \lambda (\mathbf{x}^T \Sigma \mathbf{x} - a), \tag{D2}$$

where  $\lambda$  is the Lagrange multiplier associated with the inequality constraint. The Karush-Kuhn-Tucker (KKT) conditions [66,67], which provide necessary conditions for

optimality, are given by

$$\text{Primal feasibility: } \mathbf{x}^T \Sigma \mathbf{x} \leq a,$$

$$\text{Dual feasibility: } \lambda \geq 0,$$

$$\begin{aligned} \text{Stationarity: } \nabla_x \mathcal{L}(\mathbf{x}, \lambda) &= 2\Sigma(\mathbf{x} - \mathbf{x}_b) \\ &+ 2\lambda \Sigma \mathbf{x} = 0, \end{aligned}$$

$$\text{Complementary slackness: } \lambda(\mathbf{x}^T \Sigma \mathbf{x} - a) = 0.$$

To find the optimal solution, we first set the gradient of the Lagrangian with respect to  $\mathbf{x}$  to zero (stationarity condition) and solve for  $\mathbf{x}$  in terms of  $\lambda$  and  $\mathbf{x}_b$ , i.e.,

$$(1 + \lambda)\Sigma \mathbf{x} = \Sigma \mathbf{x}_b \Rightarrow \mathbf{x} = \frac{\mathbf{x}_b}{1 + \lambda}. \tag{D3}$$

Substituting this expression into the primal feasibility condition  $\mathbf{x}^T \Sigma \mathbf{x} \leq a$ , we solve for  $\lambda$ :

$$\lambda \geq \sqrt{\frac{\mathbf{x}_b^T \Sigma \mathbf{x}_b}{a}} - 1 \geq 0, \tag{D4}$$

which implies that

$$a \leq \mathbf{x}_b^T \Sigma \mathbf{x}_b.$$

For a minimization problem with “ $\leq$ ” constraint, we consider the positive square root in the computation of  $\lambda$ . If the computed  $\lambda$  is negative, we set  $\lambda = 0$ , indicating that the constraint is not active at the optimal solution. The optimal solution to the optimization problem is obtained by substituting the value of  $\lambda$  back into the equation for  $\mathbf{x}$ . The solution must satisfy all the KKT conditions to be considered valid.

### APPENDIX E: NUMERICAL EXPERIMENTS

The results shown in the main text correspond to performing the optimization using GUROBI 11.0 from Gurobi Optimization (2023) [7]. The hardware used is an Apple M3 MAX CPU with 16 physical cores, 16 logical processors, and usage of up to 16 threads.

*Evaluation of performance with different MIP gaps.* In the main text we discuss numerical results corresponding to certain values of MIP gap. However, we limited our discussion to the TTS. Here we provide additional results with MIP

## ALGORITHM 3. Modified modularity-based spectral optimization with threshold.

---

**Input:**  $C_{\text{struct}}$ : Clean correlation matrix (Eq. (9)),  $\phi$ : threshold  
**Output:**  $\mathbf{g}$ : vector of community assignments

- 1: Initialize:  $\mathbf{g} \leftarrow$  Algorithm 1 ( $C_{\text{struct}}$ ), and  $U \leftarrow$  subgraphs whose sizes exceed  $\phi$
- 2: **while**  $U \neq \emptyset$  **do**
- 3:    $G' \leftarrow \text{pop}(U)$   $\triangleright$  Details in Sec. III B
- 4:   **if**  $|G'| \leq \phi$  **then**  $\triangleright$  Check if the community size is less than the threshold
- 5:     Update  $\mathbf{g}$  to reflect the community labels for the vertices in  $G'$
- 6:   **else**
- 7:      $(G_1, G_2) \leftarrow \text{SPECTRAL\_SPLIT}(G')$   $\triangleright$  Split  $G'$
- 8:      $U \leftarrow U \cup \{G_1, G_2\}$
- 9:   **end if**
- 10: **end while**
- 11: **return**  $\mathbf{g}$ : the final vector of community assignment to each node
- 12: **function**  $\text{SPECTRAL\_SPLIT}(G)$
- 13:   Compute the modularity matrix  $Q_c^*$  for  $G$ .  $\triangleright$  See Eq. (13)
- 14:   Compute the highest eigenvalue and eigenvector of  $Q_c^*$
- 15:    $G_1 \leftarrow$  subgraph of  $G$  with vertices in  $I_{\text{pos}} = \{i \mid \text{eigvec}[i] \geq 0\}$
- 16:    $G_2 \leftarrow$  subgraph of  $G$  with vertices in  $I_{\text{neg}} = \{i \mid \text{eigvec}[i] < 0\}$
- 17:   **return**  $(G_1, G_2)$   $\triangleright G_1$  and  $G_2$  are both non empty sets
- 18: **end Function**

---

gaps up to 0.1 in direct optimization and compare against the decomposition pipeline that uses the default MIP gap value of  $10^{-4}$  (as in the figures in the main text). Figure 7 shows the relative performance in terms of  $H$  compared to  $H_{\text{direct}}$  that corresponds to direct optimization with MIP gap  $10^{-4}$ , which is our baseline. With a MIP gap of 0.1, we observe that the decomposition pipeline not only achieves performance comparable to direct optimization but also demonstrates greater consistency and significantly faster TTS. Note that these experiments were run on an AMD EPYC 7R13 processor with eight cores, but in GUROBI we forced the optimizations to be single threaded.

*Discussion on different set of cardinality constraints.* We also conducted additional numerical simulations beyond those discussed in the main text. In the Results section, we presented findings on portfolio optimization with cardinality constraints set at  $n/2$ . Here, we provide additional results using a different set of cardinality constraints, specifically  $n/3$ ; see Fig. 8. Although the score gap is higher, the TTS is slightly elevated for all methods compared to our previous findings with constraints of  $n/2$ . Additionally, we plot the time taken by each component of the pipeline in Fig. 9. Because both preprocessing and clustering rely on matrix diagonalization operations, their complexities are  $\mathcal{O}(n^3)$ . The total time for the optimization of subproblem (green right triangles in the figure) has a qualitatively similar trend to the other two parts of the decomposition pipeline. Using Fig. 9, we find that all modules consume similar process times, indicating that there is no bottleneck in the larger pipeline.

We also obtained benchmark results with CPLEX v20.1. However, for the integer portfolio optimization problem, the standard setting needs to be modified in order to obtain a reasonable time to solution. Specifically, the `qtolin` parameter [6]—which controls the linearization of the quadratic terms in the objective function of a (mixed integer) quadratic program—needs to be turned off, because the default setting

leads to a significant increase in the TTS. However, even with this setting disabled, our approach consistently obtained much better TTS results using GUROBI, which is why we used these numbers in this study. Additionally, for the optimization problems with a single quadratic constraint [Eq. (4)], the CPLEX solver does not support the loading of the problem formulation itself unless it is first binarized. Due to the numerical instability of this approach, we again chose to only benchmark our decomposition pipeline against GUROBI.

## APPENDIX F: DERIVATION OF MARCHENKO-PASTUR DISTRIBUTION

We begin with a short proof of Wigner's surmise using Stieltjes transformation [68]. Let  $W$  be an  $n \times n$  symmetric random matrix whose entries on and above the diagonal are i.i.d. with mean 0 and variance 1. Consider the scaled matrix  $\frac{W}{\sqrt{n}}$ , and let  $\lambda_1, \dots, \lambda_n$  be its eigenvalues. The Stieltjes transform of the empirical spectral distribution of  $\frac{W}{\sqrt{n}}$  is given by

$$S_n(z) = \frac{1}{n} \sum_{i=1}^n \frac{1}{\lambda_i - z}.$$

Using the resolvent matrix, the Stieltjes transform can also be expressed as

$$S_n(z) = \frac{1}{n} \text{Tr} \left[ \left( \frac{W}{\sqrt{n}} - zI \right)^{-1} \right].$$

Let us denote by  $M$  the matrix  $W/\sqrt{n} - zI$ , which can be partitioned as

$$M = \begin{bmatrix} \frac{W_{11}}{\sqrt{n}} - z & \frac{\mathbf{g}^T}{\sqrt{n}} \\ \frac{\mathbf{g}}{\sqrt{n}} & M \end{bmatrix},$$

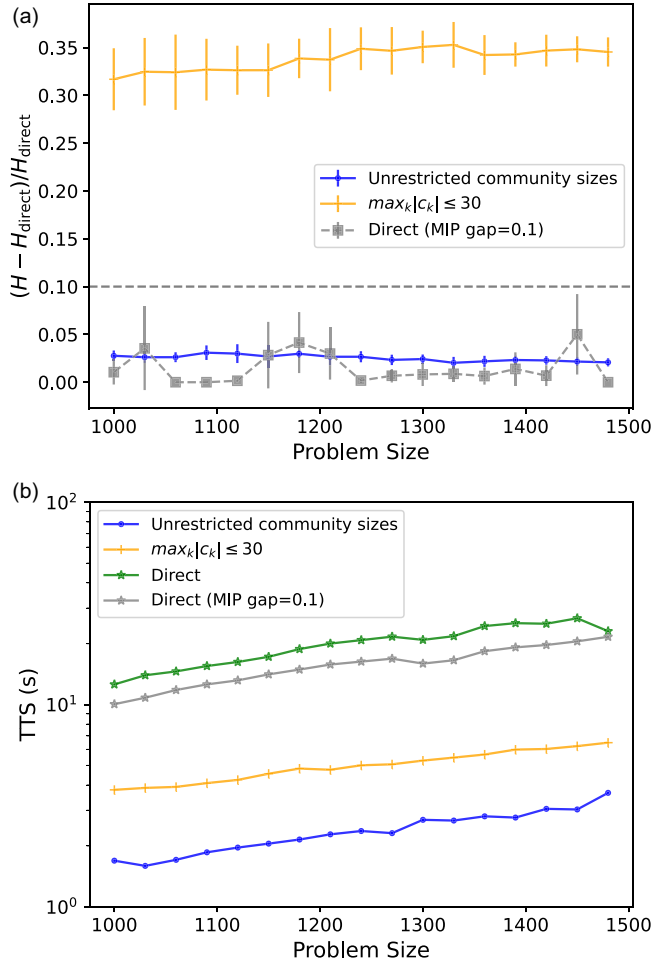


FIG. 7. Performance as a function of the problem size of the PO problem with linear cardinality constraints set at  $n/2$ . Panel (a) (top) shows the solution quality relative to the direct optimization with MIP gap  $10^{-4}$ , which is our baseline. We plot the mean values together with their error bars, corresponding to the standard deviation. Panel (b) (bottom) shows the TTS as a function of problem size. The MIP used both in direct optimization and for each of the subproblems in the decomposition pipeline is the default ( $10^{-4}$ ) unless indicated differently.

where  $\tilde{M}$  is the  $(n-1) \times (n-1)$  bottom right block of  $M$  and  $g$  is a column vector consisting of the off-diagonal elements of the first row of  $W$ . Applying the Schur complement formula for the inversion of block matrices, the (1,1) entry of  $M^{-1}$ , denoted by  $M_{11}^{-1}$ , is given by

$$M_{11}^{-1} = \left( \frac{W_{11}}{\sqrt{n}} - z - \frac{g^T}{\sqrt{n}} \tilde{M}^{-1} \frac{g}{\sqrt{n}} \right)^{-1}.$$

As  $n$  approaches infinity, the term

$$\frac{g^T}{\sqrt{n}} \tilde{M}^{-1} \frac{g}{\sqrt{n}}$$

concentrates around its expectation due to the law of large numbers. Hence, we can approximate  $M_{11}^{-1}$  by

$$\left[ \frac{W_{11}}{\sqrt{n}} - z - \frac{1}{n} \text{Tr}(\tilde{M}^{-1}) \right]^{-1},$$

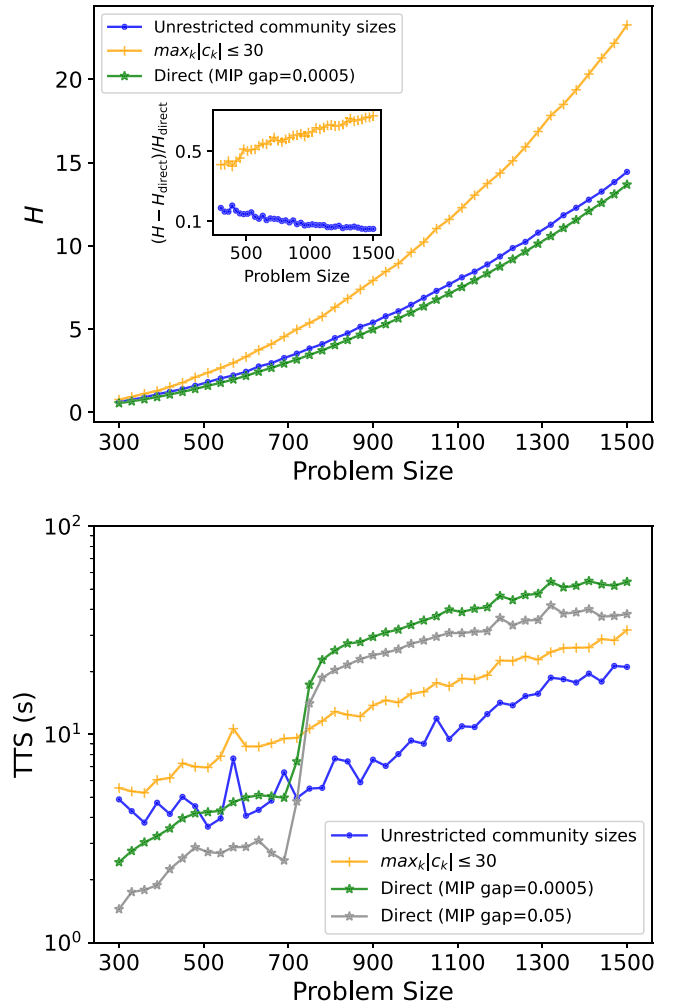


FIG. 8. Performance as a function of the problem size of the PO problem with linear cardinality constraint set at  $n/3$  [Eq. (3)]. Top panel: Solution quality  $H$ . Bottom panel: Time to solution (TTS). The line with green (gray) stars corresponds to directly optimizing with the MIP gap set at  $5 \times 10^{-4}$  ( $5 \times 10^{-2}$ ). The blue line with dots and the orange with plus signs refer to the proposed decomposition pipeline, where the former corresponds to an unrestricted community size and latter to restricting the size of the largest community size thresholded at 30. The inset plot in the top panel shows the relative drop in objective for our decomposition pipeline with and without restrictions over the community size.

which simplifies to  $-z - S_n(z)$ , assuming that the expectation of  $\text{Tr}(\tilde{M}^{-1})$  is close to  $nS_n(z)$ . Thus, the Stieltjes transform satisfies the equation

$$S_n(z) \approx \frac{1}{-z - S_n(z)}.$$

Solving this equation yields the Stieltjes transform of the semicircle law:

$$S(z) = \frac{-z \pm \sqrt{z^2 - 4}}{2}.$$

To derive the Marchenko-Pastur (MP) distribution [56], let  $X_{d \times 1}$  be a random vector uniformly distributed over the

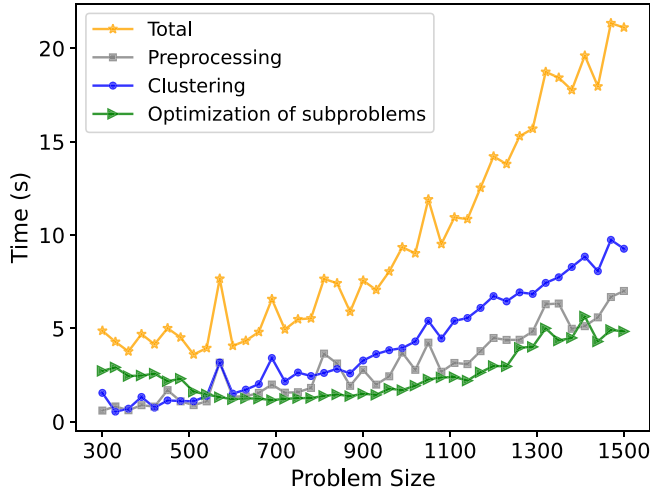


FIG. 9. Time taken (process time) for each module (preprocessing, clustering, and optimization) of the decomposition pipeline. We plot median over ten seeds of solving for portfolio optimization with assets taken from the Russell 3000 index and cardinality constraints set at  $n/3$  as a function of the problem size  $n$ . Each random seed corresponds to adding incrementally a specific pool of assets from the lower limit of problem size 300 to 1500 in step sizes of 30. The orange stars denote the total run time of the pipeline, gray squares correspond to the preprocessing, blue dots to the clustering step, and green right triangles to the optimization step.

eigenvalues  $\{\lambda_1, \dots, \lambda_d\}$ . Define the sample covariance matrix

$$\Sigma_n = \frac{1}{n} \sum_{i=1}^n X_i X_i^T,$$

which is a  $d \times d$  symmetric matrix. The Stieltjes transform of the empirical spectral distribution of  $\Sigma_n$  is

$$S_n(z) = \frac{1}{d} \text{Tr}(\Sigma_n - zI)^{-1}, \quad z \in \mathbb{C} \setminus \mathbb{R}. \quad (\text{F1})$$

To calculate  $S_n(z)$ , we consider the quantity  $\sum_{i=1}^n X_i X_i^T - nzI$ , which we denote by  $A$ . Thus, we have  $S_n(z) = (n/d) \text{Tr}(A^{-1})$ . Now, If we assume there exists a limiting spectral distribution as  $n$  grows large, we can approximate  $S_n(z)$  through  $S_{n-1}(z)$ . Let  $B = \sum_{i=1}^{n-1} X_i X_i^T - nzI$  and  $\beta = (d/n)$ . Then

$$S_{n-1}(z) = \frac{n-1}{d} \text{Tr}(B^{-1}) \approx \frac{1}{\beta} \text{Tr}(B^{-1}). \quad (\text{F2})$$

We separate out the addition of the  $n$ th term as follows:

$$A^{-1} = [B + X_n X_n^T]^{-1}.$$

Now, considering the independence of  $X_n$  from  $B$ , we use the Sherman-Morrison formula to obtain

$$X_n^T A^{-1} X_n = X_n^T (B + X_n X_n^T)^{-1} X_n = 1 - \frac{1}{1 + X_n^T B^{-1} X_n}, \quad (\text{F3})$$

which allows us to connect  $S_n(z)$  and  $S_{n-1}(z)$  through the relationship between  $A^{-1}$  and  $B^{-1}$ . We now have the basic relationship for a recursive calculation of the Stieltjes transform as  $n$  becomes large. From the independence of  $X_n$  and

$B$ , and conditioning on  $B$ , we can obtain the following for the denominator in Eq. (F3):

$$X_n^T B^{-1} X_n \approx \mathbb{E}[X_n^T B^{-1} X_n] = \text{Tr}(B^{-1}) \approx \text{Tr}(A^{-1}), \quad (\text{F4})$$

where the approximation follows from concentration considerations, and where we have also used the relationship  $\text{Tr}(A^{-1}) \approx \text{Tr}(B^{-1})$ . We therefore have the following approximation:

$$X_n^T A^{-1} X_n \approx 1 - \frac{1}{1 + \text{Tr}(A^{-1})}. \quad (\text{F5})$$

Similarly, for a different slicing of the matrix,  $X_k$ , we assume that the same approximation holds. Summing over  $n$  slices, we have

$$\sum_{k=1}^n X_k^T A^{-1} X_k \approx n \left[ 1 - \frac{1}{1 + \text{Tr}(A^{-1})} \right]. \quad (\text{F6})$$

Using the linearity and cyclic property of the trace, we can rewrite the sum as

$$\text{Tr} \left[ \sum_{k=1}^n X_k^T A^{-1} X_k \right] = \text{Tr} \left[ \sum_{k=1}^n X_k X_k^T A^{-1} \right].$$

Because  $\sum_{k=1}^n X_k X_k^T = (A + nzI)$ , we obtain

$$\text{Tr}[(A + znI)A^{-1}] = d + zn \text{Tr}(A^{-1}). \quad (\text{F7})$$

We have shown that

$$d + zn \text{Tr}(A^{-1}) \approx n \left[ 1 - \frac{1}{1 + \text{Tr}(A^{-1})} \right]. \quad (\text{F8})$$

This is a quadratic equation in  $\text{Tr}(A^{-1})$ . To solve it, we use  $s_n(\Sigma) = (1/\beta) \text{Tr}(A^{-1})$  and find

$$s_n(z) = \frac{1 - z - \beta + \sqrt{(1 - z - \beta)^2 - 4z\beta}}{2}. \quad (\text{F9})$$

This is the same as for the Stieltjes transformation of the Marchenko-Pastur distribution [68], which demonstrates uniqueness and hence proves the result that, indeed, the eigenvalue distribution of random correlation matrices follow the Marchenko-Pastur distribution. The density function of the Marchenko-Pastur distribution is given by

$$\rho_{\text{MP}}(\lambda) = \frac{1}{2\pi\beta\lambda} \sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}, \quad \lambda \in [\lambda_-, \lambda_+],$$

where  $\lambda_{\pm} = (1 \pm \sqrt{\beta})^2$ .

The Stieltjes transform  $S(z)$  of a probability distribution  $\rho(\lambda)$  is defined as

$$S(z) = \int \frac{\rho(\lambda)}{\lambda - z} d\lambda, \quad z \in \mathbb{C} \setminus \mathbb{R}.$$

For the Marchenko Pastur distribution, the Stieltjes transform  $S_{\text{MP}}(z)$  can be derived as follows:

$$S_{\text{MP}}(z) = \int_{\lambda_-}^{\lambda_+} \frac{\rho_{\text{MP}}(\lambda)}{\lambda - z} d\lambda.$$

Substituting  $\rho_{\text{MP}}(\lambda)$ , we have

$$S_{\text{MP}}(z) = \int_{\lambda_-}^{\lambda_+} \frac{1}{2\pi\beta\lambda} \sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)} \frac{1}{\lambda - z} d\lambda. \quad (\text{F10})$$

Using the Residue theorem, this integral evaluates to

$$S_{\text{MP}}(z) = \frac{1 - \beta - z + \sqrt{(z - \lambda_-)(z - \lambda_+)}}{2\beta z}. \quad (\text{F11})$$

Here, the square root is chosen such that  $\sqrt{(z - \lambda_-)(z - \lambda_+)}$  has a branch cut on the real axis from  $\lambda_-$  to  $\lambda_+$ . We have shown the derivation of the Marchenko-Pastur distribution for correlation matrices with  $\sigma^2 = 1$ .

- 
- [1] R. J. Vaessens, E. H. Aarts, and J. K. Lenstra, A local search template, *Comput. Oper. Res.* **25**, 969 (1998).
- [2] W. Wang, J. Machta, and H. G. Katzgraber, Comparing monte carlo methods for finding ground states of Ising spin glasses: Population annealing, simulated annealing, and parallel tempering, *Phys. Rev. E* **92**, 013303 (2015).
- [3] P. Hauke, H. G. Katzgraber, W. Lechner, H. Nishimori, and W. D. Oliver, Perspectives of quantum annealing: methods and implementations, *Rep. Prog. Phys.* **83**, 054401 (2020).
- [4] A. H. Land and A. G. Doig, An automatic method of solving discrete programming problems, *Econometrica* **28**, 497 (1960).
- [5] G. Cornuejols and R. Tütüncü, *Optimization Methods in Finance* (Carnegie Mellon University, Pittsburgh, 2006).
- [6] IBM, CPLEX V12.1, User's Manual, 2009, p. 157.
- [7] Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual, 2023, <https://www.gurobi.com>.
- [8] J. Clausen, Branch and bound algorithms—principles and examples (unpublished).
- [9] F. G. Brandao and K. M. Svore, in *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)* (IEEE, Piscataway, NJ, 2017), pp. 415–426.
- [10] A. Abbas, A. Ambainis, B. Augustino, A. Bärtshi, H. Buhrman, C. Coffrin, G. Cortiana, V. Dunjko, D. J. Egger, B. G. Elmegreen *et al.*, Quantum optimization: Potential, challenges, and the path forward, *Nat. Rev. Phys.* **6**, 718 (2024).
- [11] A. M. Dalzell, S. McArdle, M. Berta, P. Bienias, C.-F. Chen, A. Gilyén, C. T. Hann, M. J. Kastoryano, E. T. Khabiboulline, A. Kubica *et al.*, Quantum algorithms: A survey of applications and end-to-end complexities, [arXiv:2310.03011](https://arxiv.org/abs/2310.03011).
- [12] D. Herman, C. Googin, X. Liu, A. Galda, I. Safro, Y. Sun, M. Pistoia, and Y. Alexeev, A survey of quantum computing for finance, [arXiv:2201.02773](https://arxiv.org/abs/2201.02773).
- [13] Z. He, S. Chakrabarti, D. Herman, N. Kumar, C. Li, P. Minssen, P. Niroula, S. Ruslan, Y. Sun, S. H. Sureshbabu *et al.*, in *61st ACM/IEEE Design Automation Conference (DAC)* (IEEE, Piscataway, 2024), pp. 1–4.
- [14] T. Hogg and D. Portnov, Quantum optimization, *Inf. Sci.* **128**, 181 (2000).
- [15] E. Farhi, J. Goldstone, and S. Gutmann, A quantum approximate optimization algorithm, [arXiv:1411.4028](https://arxiv.org/abs/1411.4028).
- [16] M. P. Harrigan, K. J. Sung, M. Neeley, K. J. Satzinger, F. Arute, K. Arya, J. Atalaya, J. C. Bardin, R. Barends, S. Boixo *et al.*, Quantum approximate optimization of non-planar graph problems on a planar superconducting processor, *Nat. Phys.* **17**, 332 (2021).
- [17] E. Pelofske, A. Bärtshi, and S. Eidenbenz, Short-depth qaoa circuits and quantum annealing on higher-order ising models, *npj Quantum Inf.* **10**, 30 (2024).
- [18] R. Shaydulin, C. Li, S. Chakrabarti, M. DeCross, D. Herman, N. Kumar, J. Larson, D. Lykov, P. Minssen, Y. Sun *et al.*, Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem, *Sci. Adv.* **10**, eadm6761 (2024).
- [19] P. Niroula, R. Shaydulin, R. Yalovetzky, P. Minssen, D. Herman, S. Hu, and M. Pistoia, Constrained quantum optimization for extractive summarization on a trapped-ion quantum computer, *Sci. Rep.* **12**, 17171 (2022).
- [20] G. Buonaiuto, F. Gargiulo, G. De Pietro, M. Esposito, and M. Pota, Best practices for portfolio optimization by quantum computing, experimented on real quantum devices, *Sci. Rep.* **13**, 19434 (2023).
- [21] Z. He, R. Shaydulin, S. Chakrabarti, D. Herman, C. Li, Y. Sun, and M. Pistoia, Alignment between initial state and mixer improves QAOA performance for constrained optimization, *npj Quantum Inf.* **9**, 121 (2023).
- [22] D. Herman, R. Shaydulin, Y. Sun, S. Chakrabarti, S. Hu, P. Minssen, A. Rattew, R. Yalovetzky, and M. Pistoia, Constrained optimization via quantum Zeno dynamics, *Commun. Phys.* **6**, 219 (2023).
- [23] S. H. Sureshbabu, D. Herman, R. Shaydulin, J. Basso, S. Chakrabarti, Y. Sun, and M. Pistoia, Parameter setting in quantum approximate optimization of weighted problems, *Quantum* **8**, 1231 (2024).
- [24] J. Lang, S. Zielinski, and S. Feld, Strategic portfolio optimization using simulated, digital, and quantum annealing, *Appl. Sci.* **12**, 12288 (2022).
- [25] J. Cohen, A. Khan, and C. Alexander, Portfolio optimization of 60 stocks using classical and quantum algorithms, [arXiv:2008.08669](https://arxiv.org/abs/2008.08669).
- [26] A. W. Harrow, A. Hassidim, and S. Lloyd, Quantum algorithm for linear systems of equations, *Phys. Rev. Lett.* **103**, 150502 (2009).
- [27] A. M. Childs and N. Wiebe, Hamiltonian simulation using linear combinations of unitary operations, *Quantum Inf. Comput.* **12**, 901 (2012).
- [28] S. Chakraborty, A. Gilyén, and S. Jeffery, in *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, edited by C. Baier, I. Chatzigiannakis, P. Flocchini, and S. Leonardi (Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 2019), Vol. 132, pp. 33:1–33:14.
- [29] P. Rebentrost and S. Lloyd, Quantum computational finance: Quantum algorithm for portfolio optimization, [arXiv:1811.03975](https://arxiv.org/abs/1811.03975).
- [30] R. Yalovetzky, P. Minssen, D. Herman, and M. Pistoia, Solving linear systems on quantum hardware with hybrid HHL<sup>++</sup>, *Nat. Sci. Rep.* **14**, 20610 (2024).
- [31] A. M. Dalzell, B. D. Clader, G. Salton, M. Berta, C. Y.-Y. Lin, D. A. Bader, N. Stamatopoulos, M. J. A. Schuetz, F. G. S. L. Brandão, H. G. Katzgraber, and W. J. Zeng, End-to-end resource analysis for quantum interior-point methods and portfolio optimization, *PRX Quantum* **4**, 040325 (2023).

- [32] I. Kerenidis, A. Prakash, and D. Szilágyi, in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies* (ACM, New York, 2019), pp. 147–155.
- [33] A. Montanaro, Quantum speedup of branch-and-bound algorithms, *Phys. Rev. Res.* **2**, 013056 (2020).
- [34] S. Chakrabarti, P. Minssen, R. Yalovetzky, and M. Pistoia, Universal quantum speedup for branch-and-bound, branch-and-cut, and tree-search algorithms, [arXiv:2210.03210](https://arxiv.org/abs/2210.03210).
- [35] M. MacMahon and D. Garlaschelli, Community detection for correlation matrices, *Phys. Rev. X* **5**, 021006 (2015).
- [36] S. K. David Disatnik, Portfolio optimization using a block structure for the covariance matrix, *J. Bus. Finance Account.* **39**, 806 (2012).
- [37] L. K. Chan, J. Karceski, and J. Lakonishok, NBER Working Paper 7039, National Bureau of Economic Research, 1999, JEL No. G11, G12, <http://www.nber.org/papers/w7039>.
- [38] D. León, A. Aragón, J. Sandoval, G. Hernández, A. Arévalo, and J. Niño, Clustering algorithms for risk-adjusted portfolio construction, *Proc. Comput. Sci.* **108**, 1334 (2017).
- [39] R. Rahmaniani, T. G. Crainic, M. Gendreau, and W. Rei, The benders decomposition algorithm: A literature review, *Eur. J. Oper. Res.* **259**, 801 (2017).
- [40] J. Benders, Partitioning procedures for solving mixed-variables programming problems., *Comput. Manag. Sci.* **2**, 3 (2005).
- [41] B. S. Dees, L. Stanković, A. G. Constantinides, and D. P. Mandic, in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (IEEE, Piscataway, NJ, 2020), pp. 8454–8458.
- [42] A. Arroyo, B. Scalzo, L. Stankovic, and D. P. Mandic, Dynamic portfolio cuts: A spectral approach to graph-theoretic diversification, [arXiv:2106.03417](https://arxiv.org/abs/2106.03417).
- [43] R. Shaydulin, H. Ushijima-Mwesigwa, I. Safro, S. Mniszewski, and Y. Alexeev, Community detection across emerging quantum architectures, in *Proceedings of the 3rd International Workshop on Post Moore's Era Supercomputing (in conjunction with Supercomputing '18)*, pp. 12–14, [arXiv:1810.07765](https://arxiv.org/abs/1810.07765).
- [44] R. Shaydulin, H. Ushijima-Mwesigwa, I. Safro, S. Mniszewski, and Y. Alexeev, Network community detection on small quantum computers, *Adv. Quantum Technol.* **2**, 1900029 (2019).
- [45] R. Shaydulin, H. Ushijima-Mwesigwa, C. F. A. Negre, I. Safro, S. M. Mniszewski, and Y. Alexeev, A hybrid approach for solving optimization problems on small quantum computers, *Computer* **52**, 18 (2019).
- [46] H. Ushijima-Mwesigwa, R. Shaydulin, C. F. A. Negre, S. M. Mniszewski, Y. Alexeev, and I. Safro, Multilevel combinatorial optimization across quantum architectures, *ACM Trans. Quantum Comput.* **2**, 1 (2021).
- [47] J. Wishart, The generalised product moment distribution in samples from a normal multivariate population, *Biometrika* **20A**, 32 (1928).
- [48] J. Bun, J.-P. Bouchaud, and M. Potters, Cleaning large correlation matrices: Tools from random matrix theory, *Phys. Rep.* **666**, 1 (2017).
- [49] W. James and C. Stein, in *Proceedings of the Berkeley Symposium on Mathematical Statistics and Probability* (University of California Press, 1961), Vol. 4.1, pp. 361–379.
- [50] C. Stein, Lectures on the theory of estimation of many parameters, *J. Sov. Math.* **34**, 1373 (1986).
- [51] O. Ledoit and M. Wolf, A well-conditioned estimator for large-dimensional covariance matrices, *J. Multivariate Anal.* **88**, 365 (2004).
- [52] O. Ledoit and M. Wolf, Nonlinear shrinkage estimation of large-dimensional covariance matrices, *Ann. Stat.* **40**, 1024 (2012).
- [53] O. Ledoit and M. Wolf, Spectrum estimation: A unified framework for covariance matrix estimation and PCA in large dimensions, *J. Multivariate Anal.* **139**, 360 (2015).
- [54] O. Ledoit and M. Wolf, Numerical implementation of the quest function, *Comput. Stat. Data Anal.* **115**, 199 (2017).
- [55] O. Ledoit and M. Wolf, Analytical nonlinear shrinkage of large-dimensional covariance matrices, *Ann. Stat.* **48**, 3043 (2020).
- [56] V. A. Marčenko and L. A. Pastur, Distribution of eigenvalues for some sets of random matrices, *Math. USSR-Sb.* **1**, 457 (1967).
- [57] J. P. Bouchaud and M. Potters, Financial applications of random matrix theory: a short review, [arXiv:0910.1205](https://arxiv.org/abs/0910.1205).
- [58] J.-P. Bouchaud and M. Potters, *Theory of Financial Risk and Derivative Pricing: From Statistical Physics to Risk Management*, 2nd ed. (Cambridge University Press, Cambridge, 2003).
- [59] L. Laloux, P. Cizeau, M. Potters, and J.-P. Bouchaud, Random matrix theory and financial correlations, *Int. J. Theor. Appl. Finance* **03**, 391 (2000).
- [60] D. L. Donoho, M. Gavish, and I. M. Johnstone, Optimal shrinkage of eigenvalues in the spiked covariance model, *Ann. Stat.* **46**, 1742 (2018).
- [61] M. E. J. Newman, Modularity and community structure in networks, *Proc. Natl. Acad. Sci. USA* **103**, 8577 (2006).
- [62] C.-K. Tsung, H. Ho, S. Chou, J. Lin, and S. Lee, in *2016 International Computer Symposium (ICS)* (IEEE, Piscataway, NJ, 2016), pp. 12–17.
- [63] F. R. K. Chung, *Spectral Graph Theory*, CBMS Regional Conference Series in Mathematics No. 92 (American Mathematical Society, Providence, RI, 1997).
- [64] D. Willsch, M. Willsch, F. Jin, K. Michielsen, and H. De Raedt, GPU-accelerated simulations of quantum annealing and the quantum approximate optimization algorithm, *Comput. Phys. Commun.* **278**, 108411 (2022).
- [65] <https://github.com/jpmorganchase/dcmppln>.
- [66] W. Karush, Minima of functions of several variables with inequalities as side constraints, M.Sc. dissertation, Department of Mathematics, University of Chicago, 1939.
- [67] H. W. Kuhn and A. W. Tucker, Nonlinear programming, in *Traces and Emergence of Nonlinear Programming* (Springer, Berlin, 2013), pp. 247–258.
- [68] G. Livan, M. Novaes, and P. Vivo, *Introduction to Random Matrices* (Springer International, Cham, 2018).